

Gambella University

Faculty of Natural and Computational Sciences

Department of Computer Science

EXIT EXAM TUTORIAL MODULE

WEB PROGRAMMING

Prepared By: Lecturer Abebe Alambo (Master's Degree in Computer Science)

WEB PROGRAMMING MODULE

Date: January 03, 2023
Gambella University

Contents

CHAPTER ONE.....	1
WEB PROGRAMMING BASICS	1
HTML Document Structure	1
HTML Page Structure	2
HTML Editors	2
HTML Documents	3
HTML Headings	3
HTML Paragraphs.....	4
HTML Links	4
HTML Images	4
HTML Elements	4
HTML Attributes	5
HTML Attributes	5
HTML Headings	7
HTML Formatting Elements.....	8
HTML Links - Hyperlinks	8
HTML Tables.....	11
HTML Lists.....	11
CHAPTER TWO.....	14
CASCADING STYLE SHEET	14
How to Add CSS Web Pages	16
CSS Comments	18
CSS Backgrounds	19
CSS Borders.....	19
CSS Margins	20
CSS Padding	22
CHAPTER THREE	25
JAVASCRIPT.....	25
Overview.....	25
Adding JavaScript to Web Pages	25
Embedding the JavaScript Code	25
JavaScript Syntax.....	26
JavaScript Comments.....	27
JavaScript Variables.....	27

Data Types in JavaScript.....	29
JavaScript Operators	30
JavaScript Functions	32
CHAPTER FOUR	41
HYPERTEXT PREPROCESSOR (PHP).....	41
Common uses of PHP	41
Characteristics of PHP	41
Variable Scope	46
PHP - GET & POST Methods	55
PHP Cookies	59
PHP - Sessions	61

WEB PROGRAMMING MODULE

CHAPTER ONE

WEB PROGRAMMING BASICS

What is HTML?

- HTML stands for Hyper Text Markup Language
- HTML is the standard markup language for creating Web pages
- HTML describes the structure of a Web page
- HTML consists of a series of elements
- HTML elements tell the browser how to display the content
- HTML elements label pieces of content such as "this is a heading", "this is a paragraph", "this is a link", etc.

HTML Document Structure

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>
<h1>My First Heading</h1>
<p>My first paragraph.</p>
</body>
</html>
```

Example Explained

- The `<!DOCTYPE html>` declaration defines that this document is an HTML5 document
- The `<html>` element is the root element of an HTML page
- The `<head>` element contains meta information about the HTML page
- The `<title>` element specifies a title for the HTML page (which is shown in the browser's title bar or in the page's tab)
- The `<body>` element defines the document's body, and is a container for all the visible contents, such as headings, paragraphs, images, hyperlinks, tables, lists, etc.
- The `<h1>` element defines a large heading
- The `<p>` element defines a paragraph

What is an HTML Element?

An HTML element is defined by a start tag, some content, and an end tag:

```
<tagname>Content goes here...</tagname>
```

The HTML **element** is everything from the start tag to the end tag:

```
<h1>My First Heading</h1>
<p>My first paragraph.</p>
```

Start tag	Element content	End tag
<h1>	My First Heading	</h1>
<p>	My first paragraph.	</p>
 	<i>none</i>	<i>none</i>

Note: Some HTML elements have no content (like the `<br>` element). These elements are called empty elements. Empty elements do not have an end tag!

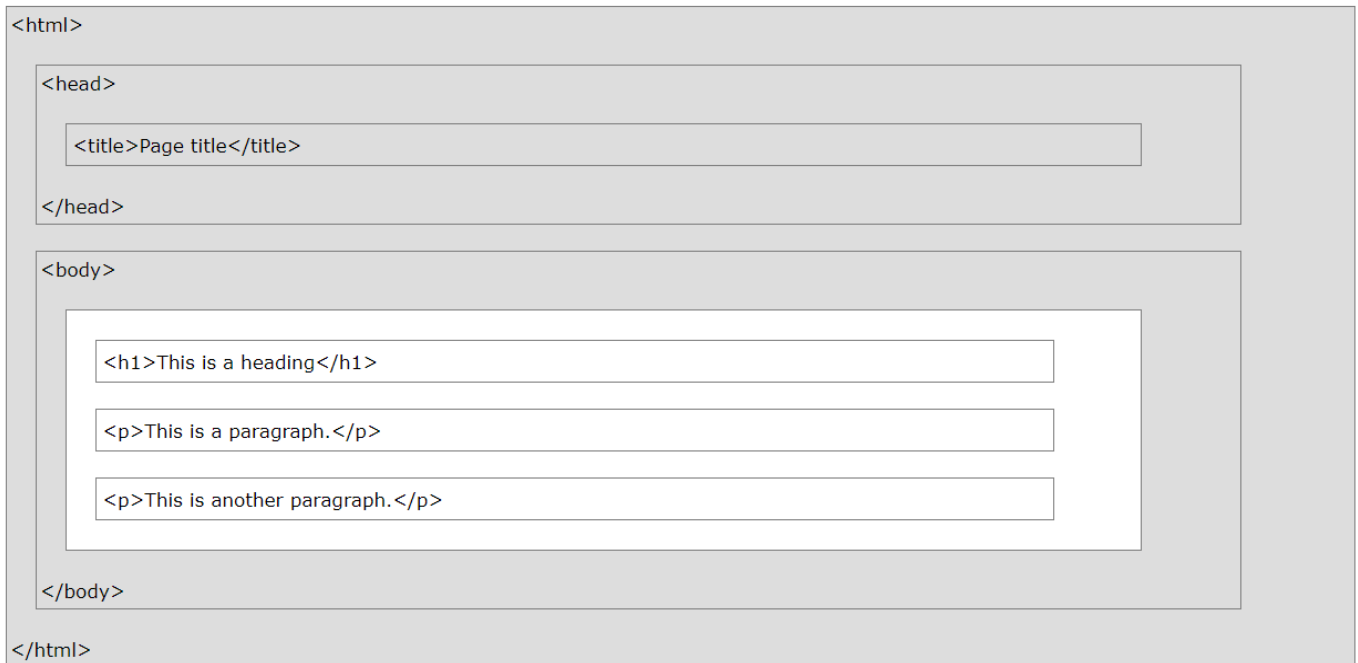
Web Browsers

The purpose of a web browser (Chrome, Edge, Firefox, and Safari) is to read HTML documents and display them correctly.

A browser does not display the HTML tags, but uses them to determine how to display the document:

HTML Page Structure

Below is a visualization of an HTML page structure:



Note: The content inside the <body> section (the white area above) will be displayed in a browser. The content inside the <title> element will be shown in the browser's title bar or in the page's tab.

HTML Editors

A simple text editor is all you need to learn HTML.

Learn HTML Using Notepad or TextEdit

Web pages can be created and modified by using professional HTML editors.

However, for learning HTML we recommend a simple text editor like Notepad (PC) or TextEdit (Mac).

We believe in that using a simple text editor is a good way to learn HTML. Follow the steps below to create your first web page with Notepad or TextEdit.

Step 1: Open Notepad (PC)

Windows 8 or later:

Open the **Start Screen** (the window symbol at the bottom left on your screen). Type **Notepad**.

Windows 7 or earlier:

Open **Start > Programs > Accessories > Notepad**

Step 2: Write Some HTML

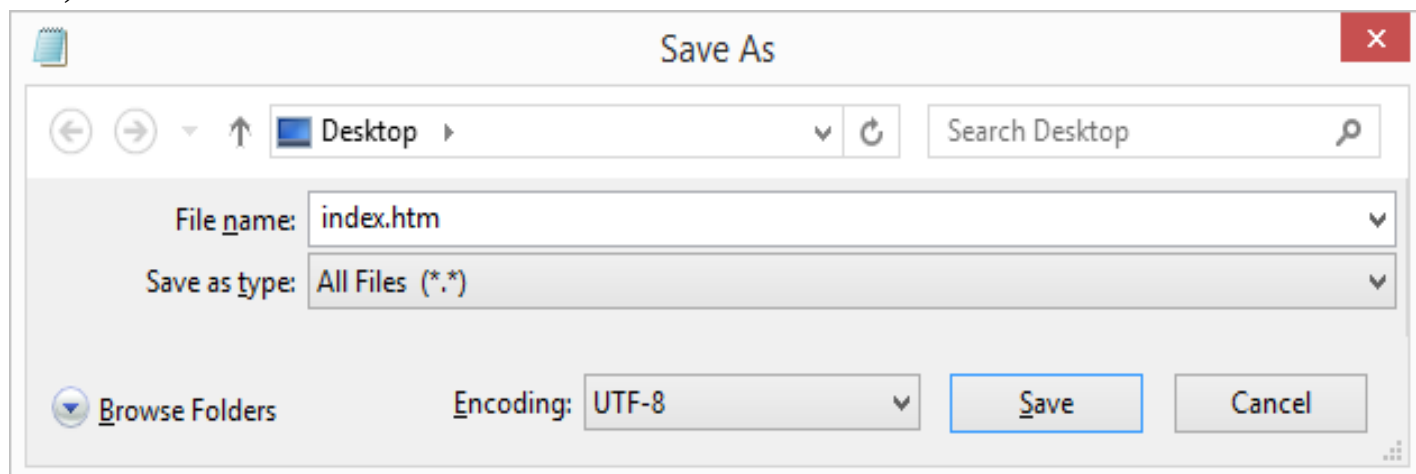
Write or copy the following HTML code into Notepad:

```
<!DOCTYPE html>
<html>
<body>
<h1>My First Heading</h1>
<p>My first paragraph.</p>
</body>
</html>
```

Step 3: Save the HTML Page

Save the file on your computer. Select **File > Save as** in the Notepad menu.

Name the file "**index.htm**" and set the encoding to **UTF-8** (which is the preferred encoding for HTML files).

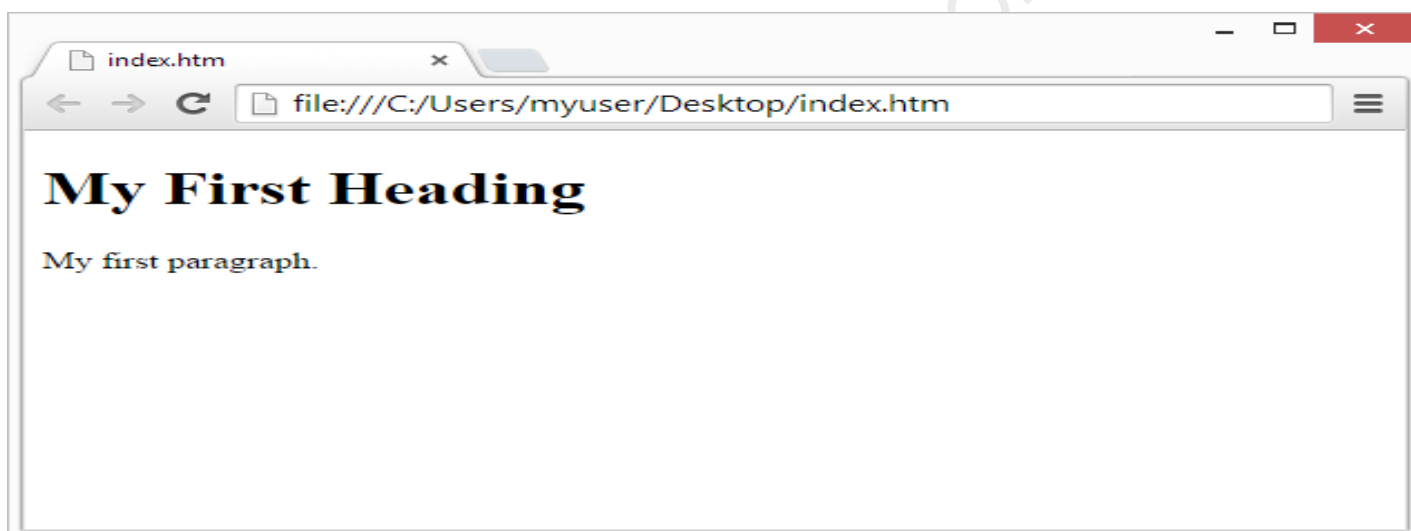


Tip: You can use either .htm or .html as file extension. There is no difference, it is up to you.

Step 4: View the HTML Page in Your Browser

Open the saved HTML file in your favorite browser (double click on the file, or right-click - and choose "Open with").

The result will look much like this:



HTML Documents

All HTML documents must start with a document type declaration: `<!DOCTYPE html>`.

The HTML document itself begins with `<html>` and ends with `</html>`.

The visible part of the HTML document is between `<body>` and `</body>`.

The `<!DOCTYPE>` Declaration

The `<!DOCTYPE>` declaration represents the document type, and helps browsers to display web pages correctly.

It must only appear once, at the top of the page (before any HTML tags).

The `<!DOCTYPE>` declaration is not case sensitive.

The `<!DOCTYPE>` declaration for HTML5 is:

```
<!DOCTYPE html>
```

HTML Headings

HTML headings are defined with the `<h1>` to `<h6>` tags.

`<h1>` defines the most important heading. `<h6>` defines the least important heading:

Example

`<h1>This is heading 1</h1>`

`<h2>This is heading 2</h2>`

`<h3>This is heading 3</h3>`

HTML Paragraphs

HTML paragraphs are defined with the `<p>` tag:

Example

`<p>This is a paragraph.</p>`

`<p>This is another paragraph.</p>`

HTML Links

HTML links are defined with the `<a>` tag:

Example

`<a href="https://www.w3schools.com">This is a link</a>`

The link's destination is specified in the `href` attribute.

Attributes are used to provide additional information about HTML elements.

You will learn more about attributes in a later chapter.

HTML Images

HTML images are defined with the `<img>` tag.

The source file (`src`), alternative text (`alt`), `width`, and `height` are provided as attributes:

HTML Elements

An HTML element is defined by a start tag, some content, and an end tag.

The HTML **element** is everything from the start tag to the end tag:

`<tagname>Content goes here...</tagname>`

Examples of some HTML elements:

`<h1>My First Heading</h1>`

`<p>My first paragraph.</p>`

Nested HTML Elements

HTML elements can be nested (this means that elements can contain other elements).

All HTML documents consist of nested HTML elements.

The following example contains four HTML elements (`<html>`, `<body>`, `<h1>` and `<p>`):

Example

`<!DOCTYPE html>`

`<html>`

`<body>`

`<h1>My First Heading</h1>`

`<p>My first paragraph.</p>`

`</body>`

`</html>`

Example Explained

The `<html>` element is the root element and it defines the whole HTML document.

It has a start tag `<html>` and an end tag `</html>`.

Then, inside the `<html>` element there is a `<body>` element:

`<body>`

`<h1>My First Heading</h1>`

`<p>My first paragraph.</p>`

`</body>`

The `<body>` element defines the document's body.

It has a start tag `<body>` and an end tag `</body>`.

Then, inside the `<body>` element there are two other elements: `<h1>` and `<p>`:

`<h1>My First Heading</h1>`

`<p>My first paragraph.</p>`

The `<h1>` element defines a heading.

It has a start tag `<h1>` and an end tag `</h1>`:

```
<h1>My First Heading</h1>
```

The `<p>` element defines a paragraph.

It has a start tag `<p>` and an end tag `</p>`:

```
<p>My first paragraph.</p>
```

Never Skip the End Tag

Some HTML elements will display correctly, even if you forget the end tag:

Example

```
<html>
```

```
<body>
```

```
<p>This is a paragraph
```

```
<p>This is a paragraph
```

```
</body>
```

```
</html>
```

However, never rely on this! Unexpected results and errors may occur if you forget the end tag!

Empty HTML Elements

HTML elements with no content are called empty elements.

The `<br>` tag defines a line break, and is an empty element without a closing tag:

Example

```
<p>This is a <br> paragraph with a line break.</p>
```

HTML is Not Case Sensitive

HTML tags are not case sensitive: `<P>` means the same as `<p>`.

The HTML standard does not require lowercase tags, but W3C **recommends** lowercase in HTML, and **demand**s lowercase for stricter document types like XHTML.

HTML Attributes

HTML attributes provide additional information about HTML elements.

HTML Attributes

- All HTML elements can have **attributes**
- Attributes provide **additional information** about elements
- Attributes are always specified in **the start tag**
- Attributes usually come in name/value pairs like: **name="value"**

The href Attribute

The `<a>` tag defines a hyperlink. The **href** attribute specifies the URL of the page the link goes to:

Example

```
<a href="https://www.w3schools.com">Visit W3Schools</a>
```

The src Attribute

The `<img>` tag is used to embed an image in an HTML page. The **src** attribute specifies the path to the image to be displayed:

Example

```

```

There are two ways to specify the URL in the **src** attribute:

Absolute URL - Links to an external image that is hosted on another website. Example:

```
src="https://www.w3schools.com/images/img_girl.jpg".
```

Notes: External images might be under copyright. If you do not get permission to use it, you may be in violation of copyright laws. In addition, you cannot control external images; it can suddenly be removed or

2. Relative URL - Links to an image that is hosted within the website. Here, the URL does not include the domain name. If the URL begins without a slash, it will be relative to the current page. Example:

```
src="img_girl.jpg". If the URL begins with a slash, it will be relative to the domain. Example:
```

```
src="/images/img_girl.jpg".
```


Tip: It is almost always best to use relative URLs. They will not break if you change domain.

The width and height Attributes

The `<img>` tag should also contain the `width` and `height` attributes, which specifies the width and height of the image (in pixels):

Example

```

```

The alt Attribute

The required `alt` attribute for the `<img>` tag specifies an alternate text for an image, if the image for some reason cannot be displayed. This can be due to slow connection, or an error in the `src` attribute, or if the user uses a screen reader.

Example

```

```

Example

See what happens if we try to display an image that does not exist:

```

```

The style Attribute

The `style` attribute is used to add styles to an element, such as color, font, size, and more.

Example

```
<p style="color:red;">This is a red paragraph.</p>
```

The lang Attribute

You should always include the `lang` attribute inside the `<html>` tag, to declare the language of the Web page. This is meant to assist search engines and browsers.

The following example specifies English as the language:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<body>
```

```
...
```

```
</body>
```

```
</html>
```

Country codes can also be added to the language code in the `lang` attribute. So, the first two characters define the language of the HTML page, and the last two characters define the country.

The following example specifies English as the language and United States as the country:

```
<!DOCTYPE html>
```

```
<html lang="en-US">
```

```
<body>
```

```
...
```

```
</body>
```

```
</html>
```

The title Attribute

The `title` attribute defines some extra information about an element.

The value of the title attribute will be displayed as a tooltip when you mouse over the element:

Example

```
<p title="I'm a tooltip">This is a paragraph.</p>
```

We Suggest: Always Use Lowercase Attributes

The HTML standard does not require lowercase attribute names.

The title attribute (and all other attributes) can be written with uppercase or lowercase like title or TITLE. However, W3C recommends lowercase attributes in HTML, and demands lowercase attributes for stricter document types like XHTML.

At W3Schools we always use lowercase attribute names.

We Suggest: Always Quote Attribute Values

The HTML standard does not require quotes around attribute values.

However, W3C recommends quotes in HTML, and demands quotes for stricter document types like XHTML.

Good:

```
<a href="https://www.w3schools.com/html/">Visit our HTML tutorial</a>
```

Bad:

```
<a href=https://www.w3schools.com/html/>Visit our HTML tutorial</a>
```

Sometimes you have to use quotes. This example will not display the title attribute correctly, because it contains a space:

Example

```
<p title>About W3Schools>
```

At W3Schools we always use quotes around attribute values.

Single or Double Quotes?

Double quotes around attribute values are the most common in HTML, but single quotes can also be used.

In some situations, when the attribute value itself contains double quotes, it is necessary to use single quotes:

```
<p title='John "ShotGun" Nelson'>
```

Or vice versa:

```
<p title="John 'ShotGun' Nelson">
```

Topic Summary

All HTML elements can have attributes

The **href** attribute of **<a>** specifies the URL of the page the link goes to

The **src** attribute of **** specifies the path to the image to be displayed

The **width** and **height** attributes of **** provide size information for images

The **alt** attribute of **** provides an alternate text for an image

The **style** attribute is used to add styles to an element, such as color, font, size, and more

The **lang** attribute of the **<html>** tag declares the language of the Web page

HTML Headings

HTML headings are titles or subtitles that you want to display on a webpage.

Example

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6

HTML headings are defined with the **<h1>** to **<h6>** tags.

<h1> defines the most important heading. **<h6>** defines the least important heading.

Example

```
<h1>Heading 1</h1>
```

```
<h2>Heading 2</h2>
```

```
<h3>Heading 3</h3>
```

```
<h4>Heading 4</h4>
```

```
<h5>Heading 5</h5>
```

```
<h6>Heading 6</h6>
```

Note: Browsers automatically add some white space (a margin) before and after a heading.

Headings Are Important: Search engines use the headings to index the structure and content of your web pages.

Users often skim a page by its headings. It is important to use headings to show the document structure.

`<h1>` headings should be used for main headings, followed by `<h2>` headings, then the less important `<h3>`, and so on.

Note: Use HTML headings for headings only. Don't use headings to make text **BIG** or **bold**.

Bigger Headings

Each HTML heading has a default size. However, you can specify the size for any heading with the `style` attribute, using the CSS `font-size` property:

Example

```
<h1 style="font-size:60px;">Heading 1</h1>
```

HTML Formatting Elements

Formatting elements were designed to display special types of text:

`<b>` - Bold text

`<strong>` - Important text

`<i>` - Italic text

`<em>` - Emphasized text

`<mark>` - Marked text

`<small>` - Smaller text

`<del>` - Deleted text

`<ins>` - Inserted text

`<sub>` - Subscript text

`<sup>` - Superscript text

HTML `<b>` and `<strong>` Elements

The HTML `<b>` element defines bold text, without any extra importance.

HTML Links - Hyperlinks

HTML links are hyperlinks.

You can click on a link and jump to another document.

When you move the mouse over a link, the mouse arrow will turn into a little hand.

Note: A link does not have to be text. A link can be an image or any other HTML element!

HTML Links - Syntax

The HTML `<a>` tag defines a hyperlink. It has the following syntax:

```
<a href="url">link text</a>
```

The most important attribute of the `<a>` element is the `href` attribute, which indicates the link's destination.

The *link text* is the part that will be visible to the reader.

Clicking on the link text, will send the reader to the specified URL address.

Example

This example shows how to create a link to W3Schools.com:

```
<a href="https://www.w3schools.com/">Visit W3Schools.com!</a>
```

By default, links will appear as follows in all browsers:

An unvisited link is underlined and blue

A visited link is underlined and purple

An active link is underlined and red

Tip: Links can of course be styled with CSS, to get another look!

HTML Links - The target Attribute

By default, the linked page will be displayed in the current browser window. To change this, you must specify another target for the link.

The `target` attribute specifies where to open the linked document.

The **target** attribute can have one of the following values:

_self - Default. Opens the document in the same window/tab as it was clicked

_blank - Opens the document in a new window or tab

_parent - Opens the document in the parent frame

_top - Opens the document in the full body of the window

Example

Use target="_blank" to open the linked document in a new browser window or tab:

```
<a href="https://www.w3schools.com/" target="_blank">Visit W3Schools!</a>
```

Absolute URLs vs. Relative URLs

Both examples above are using an **absolute URL** (a full web address) in the **href** attribute.

A local link (a link to a page within the same website) is specified with a **relative URL** (without the "https://www" part):

Example

```
<h2>Absolute URLs</h2>
```

```
<p><a href="https://www.w3.org/">W3C</a></p>
```

```
<p><a href="https://www.google.com/">Google</a></p>
```

```
<h2>Relative URLs</h2>
```

```
<p><a href="html_images.asp">HTML Images</a></p>
```

```
<p><a href="/css/default.asp">CSS Tutorial</a></p>
```

HTML Links - Use an Image as a Link

To use an image as a link, just put the **** tag inside the **<a>** tag:

Example

```
<a href="default.asp">
```

```

```

```
</a>
```

Link to an Email Address

Use **mailto:** inside the **href** attribute to create a link that opens the user's email program (to let them send a new email):

Example

```
<a href="mailto:someone@example.com">Send email</a>
```

Button as a Link

To use an HTML button as a link, you have to add some JavaScript code.

JavaScript allows you to specify what happens at certain events, such as a click of a button:

Example

```
<button onclick="document.location='default.asp'">HTML Tutorial</button>
```

Link Titles

The **title** attribute specifies extra information about an element. The information is most often shown as a tooltip text when the mouse moves over the element.

Example

```
<a href="https://www.w3schools.com/html/" title="Go to W3Schools HTML section">Visit our HTML Tutorial</a>
```

More on Absolute URLs and Relative URLs

Example

Use a full URL to link to a web page:

```
<a href="https://www.w3schools.com/html/default.asp">HTML tutorial</a>
```

Example

Link to a page located in the html folder on the current web site:

```
<a href="/html/default.asp">HTML tutorial</a>
```

Example

Link to a page located in the same folder as the current page:

```
<a href="default.asp">HTML tutorial</a>
```

Topic Summary

Use the `<a>` element to define a link

Use the `href` attribute to define the link address

Use the `target` attribute to define where to open the linked document

Use the `<img>` element (inside `<a>`) to use an image as a link

Use the `mailto:` scheme inside the `href` attribute to create a link that opens the user's email program

HTML Images Syntax

The HTML `<img>` tag is used to embed an image in a web page.

Images are not technically inserted into a web page; images are linked to web pages. The `<img>` tag creates a holding space for the referenced image.

The `<img>` tag is empty, it contains attributes only, and does not have a closing tag.

The `<img>` tag has two required attributes:

`src` - Specifies the path to the image

`alt` - Specifies an alternate text for the image

Syntax

```

```

The src Attribute

The required `src` attribute specifies the path (URL) to the image.

Note: When a web page loads; it is the browser, at that moment that gets the image from a web server and inserts it into the page. Therefore, make sure that the image actually stays in the same spot in relation to the web page, otherwise your visitors will get a broken link icon. The broken link icon and the alt text are shown if the browser cannot find the image.

The alt Attribute

The required `alt` attribute provides an alternate text for an image, if the user for some reason cannot view it (because of slow connection, an error in the `src` attribute, or if the user uses a screen reader).

The value of the `alt` attribute should describe the image:

Example

```

```

Common Image Formats

Here are the most common image file types, which are supported in all browsers (Chrome, Edge, Firefox, Safari, Opera):

Abbreviation	File Format	File Extension
APNG	Animated Portable Network Graphics	.apng
GIF	Graphics Interchange Format	.gif
ICO	Microsoft Icon	.ico, .cur
JPEG	Joint Photographic Expert Group image	.jpg, .jpeg, .jfif, .pjpeg, .jpp
PNG	Portable Network Graphics	.png
SVG	Scalable Vector Graphics	.svg

Topic Summary

Use the HTML `<img>` element to define an image

Use the HTML `src` attribute to define the URL of the image

Use the HTML `alt` attribute to define an alternate text for an image, if it cannot be displayed

Use the HTML **width** and **height** attributes or the CSS **width** and **height** properties to define the size of the image

Use the CSS **float** property to let the image float to the left or to the right

HTML Image Tags

Tag	Description
<code></code>	Defines an image
<code><map></code>	Defines an image map
<code><area></code>	Defines a clickable area inside an image map
<code><picture></code>	Defines a container for multiple image resources

HTML Tables

HTML tables allow web developers to arrange data into rows and columns.

Define an HTML Table

The `<table>` tag defines an HTML table.

Each table row is defined with a `<tr>` tag. Each table header is defined with a `<th>` tag. Each table data/cell is defined with a `<td>` tag.

By default, the text in `<th>` elements are bold and centered.

By default, the text in `<td>` elements are regular and left-aligned.

Note: The `<td>` elements are the data containers of the table. They can contain all sorts of HTML elements; text, images, lists, other tables, etc.

Topic Summary

- ✓ Use the HTML `<table>` element to define a table
- ✓ Use the HTML `<tr>` element to define a table row
- ✓ Use the HTML `<td>` element to define a table data
- ✓ Use the HTML `<th>` element to define a table heading
- ✓ Use the HTML `<caption>` element to define a table caption
- ✓ Use the CSS **border** property to define a border
- ✓ Use the CSS **border-collapse** property to collapse cell borders
- ✓ Use the CSS **padding** property to add padding to cells
- ✓ Use the CSS **text-align** property to align cell text
- ✓ Use the CSS **border-spacing** property to set the spacing between cells
- ✓ Use the **colspan** attribute to make a cell span many columns
- ✓ Use the **rowspan** attribute to make a cell span many rows
- ✓ Use the **id** attribute to uniquely define one table

HTML Lists

HTML lists allow web developers to group a set of related items in lists.

Example

An unordered HTML list:	An ordered HTML list:
• Item • Item • Item • Item	1. First item 2. Second item 3. Third item 4. Fourth item

Unordered HTML List

An unordered list starts with the `<ul>` tag. Each list item starts with the `<li>` tag.

The list items will be marked with bullets (small black circles) by default:

Example

```
<ul>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

Ordered HTML List

An ordered list starts with the `<ol>` tag. Each list item starts with the `<li>` tag.

The list items will be marked with numbers by default:

Example

```
<ol>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

HTML Description Lists

HTML also supports description lists.

A description list is a list of terms, with a description of each term.

The `<dl>` tag defines the description list, the `<dt>` tag defines the term (name), and the `<dd>` tag describes each term:

Example

```
<dl>
  <dt>Coffee</dt>
  <dd>- black hot drink</dd>
  <dt>Milk</dt>
  <dd>- white cold drink</dd>
</dl>
```

HTML List Tags

Tag	Description
<code></code>	Defines an unordered list
<code></code>	Defines an ordered list
<code></code>	Defines a list item
<code><dl></code>	Defines a description list
<code><dt></code>	Defines a term in a description list
<code><dd></code>	Describes the term in a description list

HTML class Attribute

The HTML `class` attribute is used to specify a class for an HTML element.

Multiple HTML elements can share the same class.

Using the class Attribute

The `class` attribute is often used to point to a class name in a style sheet. It can also be used by a JavaScript to access and manipulate elements with the specific class name.

In the following example we have three `<div>` elements with a `class` attribute with the value of "city". All of the three `<div>` elements will be styled equally according to the `.city` style definition in the head section:

HTML id Attribute

The HTML `id` attribute is used to specify a unique id for an HTML element.

You cannot have more than one element with the same id in an HTML document.

Using the id Attribute

The id attribute specifies a unique id for an HTML element. The value of the id attribute must be unique within the HTML document.

The id attribute is used to point to a specific style declaration in a style sheet. It is also used by JavaScript to access and manipulate the element with the specific id.

The syntax for id is: write a hash character (#), followed by an id name. Then, define the CSS properties within curly braces {}.

In the following example we have an `<h1>` element that points to the id name "myHeader".

This `<h1>` element will be styled according to the `#myHeader` style definition in the head section:

HTML Iframes

An HTML iframe is used to display a web page within a web page.

HTML Iframe Syntax

The HTML `<iframe>` tag specifies an inline frame.

An inline frame is used to embed another document within the current HTML document.

Syntax

```
<iframe src="url" title="description">
```

Iframe - Set Height and Width

Use the `height` and `width` attributes to specify the size of the iframe.

The height and width are specified in pixels by default:

Example

```
<iframe src="demo_iframe.htm" height="200" width="300" title="Iframe Example"></iframe>
```

Iframe - Target for a Link

An iframe can be used as the target frame for a link.

The `target` attribute of the link must refer to the `name` attribute of the iframe:

Example

```
<iframe src="demo_iframe.htm" name="iframe_a" title="Iframe Example"></iframe>
<p><a href="https://www.w3schools.com" target="iframe_a">W3Schools.com</a></p>
```

Topic Summary

The HTML `<iframe>` tag specifies an inline frame

The `src` attribute defines the URL of the page to embed

Always include a `title` attribute (for screen readers)

The `height` and `width` attributes specifies the size of the iframe

Use `border: none;` to remove the border around the iframe

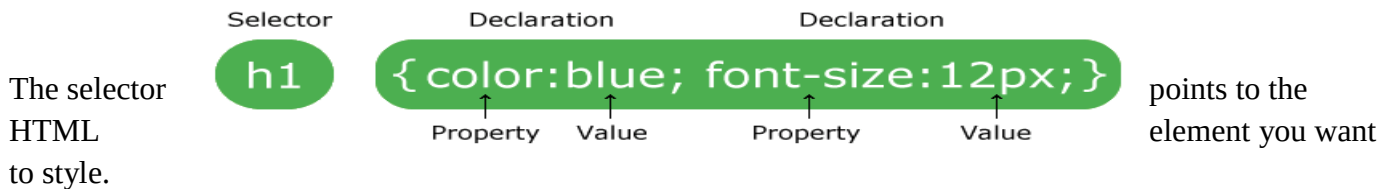
CHAPTER TWO

CASCADING STYLE SHEET

What is CSS?

- CSS stands for Cascading Style Sheets
- CSS describes how HTML elements are to be displayed on screen, paper, or in other media
- CSS is the language we use to style a Web page.
- CSS saves a lot of work. It can control the layout of multiple web pages all at once
- External stylesheets are stored in CSS files

CSS Syntax



The selector HTML to style.

The declaration block contains one or more declarations separated by semicolons.

Each declaration includes a CSS property name and a value, separated by a colon.

Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces.

Example

In this example all <p> elements will be center-aligned, with a red text color:

```
p {  
  color: red;  
  text-align: center;  
}
```

Example Explained

p is a selector in CSS (it points to the HTML element you want to style: <p>).

color is a property, and **red** is the property value

text-align is a property, and **center** is the property value

CSS Selectors

A CSS selector selects the HTML element(s) you want to style.

CSS selectors are used to "find" (or select) the HTML elements you want to style.

We can divide CSS selectors into **five** categories:

Simple selectors (select elements based on name, id, class)

Combinator selectors (select elements based on a specific relationship between them)

Pseudo-class selectors (select elements based on a certain state)

Pseudo-elements selectors (select and style a part of an element)

Attribute selectors (select elements based on an attribute or attribute value)

The CSS element Selector

The element selector selects HTML elements based on the element name.

Example

Here, all <p> elements on the page will be center-aligned, with a red text color:

```
p {  
  text-align: center;  
  color: red;  
}
```

The CSS id Selector: The id selector uses the id attribute of an HTML element to select a specific element. The id of an element is unique within a page, so the id selector is used to select one unique element!. To select an element with a specific id, write a hash (#) character, followed by the id of the element.

Example

The CSS rule below will be applied to the HTML element with id="para1":

```
#para1 {  
  text-align: center;  
  color: red;  
}
```

The CSS class Selector: The class selector selects HTML elements with a specific class attribute. To select elements with a specific class, write a period (.) character, followed by the class name.

Example

In this example all HTML elements with class="center" will be red and center-aligned:

```
.center {  
  text-align: center;  
  color: red;  
}
```

The CSS Universal Selector: The universal selector (*) selects all HTML elements on the page.

Example

The CSS rule below will affect every HTML element on the page:

```
* {  
  text-align: center;  
  color: blue;  
}
```

The CSS Grouping Selector: The grouping selector selects all the HTML elements with the same style definitions.

Look at the following CSS code (the h1, h2, and p elements have the same style definitions):

```
h1 {  
  text-align: center;  
  color: red;  
}  
h2 {  
  text-align: center;  
  color: red;  
}  
p {  
  text-align: center;  
  color: red;  
}
```

It will be better to group the selectors, to minimize the code.

To group selectors, separate each selector with a comma.

Example

In this example we have grouped the selectors from the code above:

```
h1, h2, p {  
  text-align: center;  
  color: red;  
}
```

All CSS Simple Selectors →

Selector	Example	Example description
<u>#id</u>	#firstname	Selects the element with id="firstname"
<u>.class</u>	.intro	Selects all elements with class="intro"
<u>element.class</u>	p.intro	Selects only <p> elements with class="intro"
<u>*</u>	*	Selects all elements
<u>element</u>	p	Selects all <p> elements
<u>element,element...</u>	div, p	Selects all <div> elements and all <p> elements

How to Add CSS Web Pages

When a browser reads a style sheet, it will format the HTML document according to the information in the style sheet.

Three Ways to Insert CSS

There are three ways of inserting a style sheet:

External CSS

Internal CSS

Inline CSS

External CSS: With an external style sheet, you can change the look of an entire website by changing just one file! Each HTML page must include a reference to the external style sheet file inside the <link> element, inside the head section.

Example

External styles are defined within the <link> element, inside the <head> section of an HTML page:

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="mystyle.css">
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

An external style sheet can be written in any text editor, and must be saved with a .css extension. The external .css file should not contain any HTML tags. Here is how the "mystyle.css" file looks:

"mystyle.css"

```
body {
  background-color: lightblue;
}
h1 {
  color: navy;
  margin-left: 20px;
}
```

Internal CSS: An internal style sheet may be used if one single HTML page has a unique style. The internal style is defined inside the <style> element, inside the head section.

Example

Internal styles are defined within the <style> element, inside the <head> section of an HTML page:

```
<!DOCTYPE html>
<html><head><style>
body {
  background-color: linen;
}
h1 {
  color: maroon;
  margin-left: 40px;
}
</style>
</head>
```

```

<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>

```

Inline CSS

An inline style may be used to apply a unique style for a single element.

To use inline styles, add the style attribute to the relevant element. The style attribute can contain any CSS property.

Example

Inline styles are defined within the "style" attribute of the relevant element:

```

<!DOCTYPE html>
<html>
<body>
<h1 style="color:blue;text-align:center;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>

</body>
</html>

```

Tip: An inline style loses many of the advantages of a style sheet (by mixing content with presentation). Use this method sparingly.

Multiple Style Sheets

If some properties have been defined for the same selector (element) in different style sheets, the value from the last read style sheet will be used.

Assume that an **external style sheet** has the following style for the <h1> element:

```

h1 {
  color: navy;
}

```

Then, assume that an internal style sheet also has the following style for the <h1> element:

```

h1 {
  color: orange;
}

```

Example

If the internal style is defined after the link to the external style sheet, the <h1> elements will be "orange":

```

<head>
<link rel="stylesheet" type="text/css" href="mystyle.css">
<style>
h1 {
  color: orange;
}
</style>
</head>

```

Example

However, if the internal style is defined **before** the link to the external style sheet, the <h1> elements will be "navy":

```

<head>
<style>
h1 {

```

```

color: orange;
}
</style>
<link rel="stylesheet" type="text/css" href="mystyle.css">
</head>

```

Cascading Order

What style will be used when there is more than one style specified for an HTML element?

All the styles in a page will "cascade" into a new "virtual" style sheet by the following rules, where number one has the highest priority:

Inline style (inside an HTML element)

External and internal style sheets (in the head section)

Browser default

So, an inline style has the highest priority, and will override external and internal styles and browser defaults.

CSS Comments

CSS comments are not displayed in the browser, but they can help document your source code.

Comments are used to explain the code, and may help when you edit the source code at a later date.

Comments are ignored by browsers.

A CSS comment is placed inside the `<style>` element, and starts with `/*` and ends with `*/`:

Example

```
/* This is a single-line comment */
```

```

p {
  color: red;
}

```

You can add comments wherever you want in the code:

Example

```

p {
  color: red; /* Set text color to red */
}

```

Comments can also span multiple lines:

Example

```
/* This is
a multi-line
comment */
```

```

p {
  color: red;
}

```

HTML and CSS Comments

From the HTML tutorial, you learned that you can add comments to your HTML source by using the `<!--...-->` syntax.

In the following example, we use a combination of HTML and CSS comments:

Example

```

<!DOCTYPE html>
<html>
<head>
<style>
p {
  color: red; /* Set text color to red */
}
</style>
</head>
<body>

```

```

<h2>My Heading</h2>
<!-- These paragraphs will be red -->
<p>Hello World!</p>
<p>This paragraph is styled with CSS.</p>
<p>CSS comments are not shown in the output.</p>
</body>
</html>

```

CSS Backgrounds

The CSS background properties are used to add background effects for elements.

In these chapters, you will learn about the following CSS background properties:

background-color

background-image

background-repeat

background-attachment

background-position

background (shorthand property)

CSS background-color

The **background-color** property specifies the background color of an element.

Example

The background color of a page is set like this:

```

body {
  background-color: lightblue;
}

```

CSS Borders

The CSS border properties allow you to specify the style, width, and color of an element's border.

I have borders on all sides.

I have a red bottom border.

I have rounded borders.

I have a blue left border.

CSS Border Style

The **border-style** property specifies what kind of border to display.

The following values are allowed:

- **dotted** - Defines a dotted border
- **dashed** - Defines a dashed border
- **solid** - Defines a solid border
- **double** - Defines a double border
- **groove** - Defines a 3D grooved border. The effect depends on the border-color value
- **ridge** - Defines a 3D ridged border. The effect depends on the border-color value
- **inset** - Defines a 3D inset border. The effect depends on the border-color value
- **outset** - Defines a 3D outset border. The effect depends on the border-color value
- **none** - Defines no border
- **hidden** - Defines a hidden border

The **border-style** property can have from one to four values (for the top border, right border, bottom border, and the left border).

Example

Demonstration of the different border styles:

```

p.dotted {border-style: dotted;}
p.dashed {border-style: dashed;}
p.solid {border-style: solid;}
p.double {border-style: double;}
p.groove {border-style: groove;}

```

```

p.ridge {border-style: ridge;}
p.inset {border-style: inset;}
p.outset {border-style: outset;}
p.none {border-style: none;}
p.hidden {border-style: hidden;}
p.mix {border-style: dotted dashed solid double;}

```

Result:

A dotted border.

A dashed border.

A solid border.

A double border.

A groove border. The effect depends on the border-color value.

A ridge border. The effect depends on the border-color value.

An inset border. The effect depends on the border-color value.

An outset border. The effect depends on the border-color value.

No border.

A hidden border.

A mixed border.

CSS Margins

Margins are used to create space around elements, outside of any defined borders.

This element has a margin of 70px.

CSS Margins: The CSS **margin** properties are used to create space around elements, outside of any defined borders. With CSS, you have full control over the margins. There are properties for setting the margin for each side of an element (top, right, bottom, and left).

Margin - Individual Sides: CSS has properties for specifying the margin for each side of an element:

- **margin-top**
- **margin-right**
- **margin-bottom**
- **margin-left**

All the margin properties can have the following values:

- *length* - specifies a margin in px, pt, cm, etc.
- *%* - specifies a margin in % of the width of the containing element
- *inherit* - specifies that the margin should be inherited from the parent element

Tip: Negative values are allowed.

Example

Set different margins for all four sides of a <p> element:

```

p {
  margin-top: 100px;

```

```
margin-bottom: 100px;
margin-right: 150px;
margin-left: 80px;
}
```

Margin - Shorthand Property: To shorten the code, it is possible to specify all the margin properties in one property. The **margin** property is a shorthand property for the following individual margin properties:

```
margin-top
margin-right
margin-bottom
margin-left
```

So, here is how it works:

If the **margin** property has four values:

- **margin: 25px 50px 75px 100px;**
 - top margin is 25px
 - right margin is 50px
 - bottom margin is 75px
 - left margin is 100px

Example

Use the margin shorthand property with four values:

```
p {
  margin: 25px 50px 75px 100px;
}
```

If the **margin** property has three values:

- **margin: 25px 50px 75px;**
 - top margin is 25px
 - right and left margins are 50px
 - bottom margin is 75px

Example

Use the margin shorthand property with three values:

```
p {
  margin: 25px 50px 75px;
}
```

If the **margin** property has two values:

- **margin: 25px 50px;**
 - top and bottom margins are 25px
 - right and left margins are 50px

Example

Use the margin shorthand property with two values:

```
p {
  margin: 25px 50px;
}
```

If the **margin** property has one value:

- **margin: 25px;**
 - all four margins are 25px

Example

Use the margin shorthand property with one value:

```
p {
  margin: 25px;
}
```

The auto Value

You can set the margin property to **auto** to horizontally center the element within its container.

The element will then take up the specified width, and the remaining space will be split equally between the left and right margins.

Example

Use margin: auto:

```
div {  
  width: 300px;  
  margin: auto;  
  border: 1px solid red;  
}
```

The inherit Value

This example lets the left margin of the <p class="ex1"> element be inherited from the parent element (<div>):

Example

Use of the inherit value:

```
div {  
  border: 1px solid red;  
  margin-left: 100px;  
}  
p.ex1 {  
  margin-left: inherit;  
}
```

CSS Padding

Padding is used to create space around an element's content, inside of any defined borders.

This element has a padding of 70px.

CSS Padding

The CSS **padding** properties are used to generate space around an element's content, inside of any defined borders.

With CSS, you have full control over the padding. There are properties for setting the padding for each side of an element (top, right, bottom, and left).

Padding - Individual Sides

CSS has properties for specifying the padding for each side of an element:

padding-top

padding-right

padding-bottom

padding-left

All the padding properties can have the following values:

length - specifies a padding in px, pt, cm, etc.

% - specifies a padding in % of the width of the containing element

inherit - specifies that the padding should be inherited from the parent element

Note: Negative values are not allowed.

Example

Set different padding for all four sides of a <div> element:

```
div {  
  padding-top: 50px;  
  padding-right: 30px;  
  padding-bottom: 50px;  
  padding-left: 80px;  
}
```

Padding - Shorthand Property

To shorten the code, it is possible to specify all the padding properties in one property.

The **padding** property is a shorthand property for the following individual padding properties:

padding-right

padding-bottom

padding-left

So, here is how it works:

If the **padding** property has four values:

padding: 25px 50px 75px 100px;

top padding is 25px

right padding is 50px

bottom padding is 75px

left padding is 100px

Example

Use the padding shorthand property with four values:

```
div {  
  padding: 25px 50px 75px 100px;  
}
```

If the **padding** property has three values:

padding: 25px 50px 75px;

top padding is 25px

right and left paddings are 50px

bottom padding is 75px

Example

Use the padding shorthand property with three values:

```
div {  
  padding: 25px 50px 75px;  
}
```

If the **padding** property has two values:

padding: 25px 50px;

top and bottom paddings are 25px

right and left paddings are 50px

Example

Use the padding shorthand property with two values:

```
div {  
  padding: 25px 50px;  
}
```

If the **padding** property has one value:

padding: 25px;

all four paddings are 25px

Example

Use the padding shorthand property with one value:

```
div {  
  padding: 25px;  
}
```

Padding and Element Width

The CSS **width** property specifies the width of the element's content area. The content area is the portion inside the padding, border, and margin of an element (the box model).

So, if an element has a specified width, the padding added to that element will be added to the total width of the element. This is often an undesirable result.

Example

Here, the <div> element is given a width of 300px. However, the actual width of the <div> element will be 350px (300px + 25px of left padding + 25px of right padding):

```
div {  
  width: 300px;  
  padding: 25px;  
}
```

To keep the width at 300px, no matter the amount of padding, you can use the **box-sizing** property. This causes the element to maintain its width; if you increase the padding, the available content space will decrease.

Example

Use the box-sizing property to keep the width at 300px, no matter the amount of padding:

```
div{  
  width: 300px;  
  padding: 25px;  
  box-sizing: border-box;  
}
```

WEB PROGRAMMING MODULE

CHAPTER THREE

JAVASCRIPT

Overview

Here, you will learn how easy it is to add interactivity to a web page using JavaScript. But, before we begin, make sure that you have some working knowledge of HTML and CSS. Adding JavaScript to Your Web Pages

There are typically three ways to add JavaScript to a web page: -

- ✓ Embedding the JavaScript code between a pair of `<script>` and `</script>` tag.
- ✓ Creating an external JavaScript file with the `.js` extension and then load it within the page through the `src` attribute of the `<script>` tag.
- ✓ Placing the JavaScript code directly inside an HTML tag using the special tag attributes such as `onclick`, `onmouseover`, `onkeypress`, `onload`, etc. The following sections will describe each of these procedures in detail:

Adding JavaScript to Web Pages

There are typically three ways to add JavaScript to a web page:

- ✓ Embedding the JavaScript code between a pair of `<script>` and `</script>` tag.
- ✓ Creating an external JavaScript file with the `.js` extension and then load it within the page through the `src` attribute of the `<script>` tag.
- ✓ Placing the JavaScript code directly inside an HTML tag using the special tag attributes such as `onclick`, `onmouseover`, `onkeypress`, `onload`, etc.

The following sections will describe each of these procedures in detail:

Embedding the JavaScript Code

You can embed the JavaScript code directly within your web pages by placing it between the `<script>` and `</script>` tags. The `<script>` tag indicates the browser that the contained statements are to be interpreted as executable script and not HTML. Here's an example:

```
<!DOCTYPE html>
<html lang="en">
<head> <meta charset="UTF-8"> <title>Embedding JavaScript</title> </head> <body>
<script>
var greet = "Hello World!";
document.write(greet); // Prints: Hello World!
</script>
</body></html>
```

The JavaScript code in the above example will simply prints a text message on the web page. You will learn what each of these JavaScript statements means in upcoming chapters.

Calling an External JavaScript File

You can also place your JavaScript code into a separate file with a `.js` extension, and then call that file in your document through the `src` attribute of the `<script>` tag, like this:

```
<script src="js/hello.js"></script>
```

This is useful if you want the same scripts available to multiple documents. It saves you from repeating the same task over and over again, and makes your website much easier to maintain. Well, let's create a JavaScript file named "hello.js" and place the following code in it:

```
//A function to display a message
function sayHello()
{
alert("Hello World!");
} // Call function on click of the button
document.getElementById("myBtn").onclick = sayHello;
```

Now, you can call this external JavaScript file within a web page using the `<script>` tag, like this:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8"> <title>Including External JavaScript File</title> </head>
<body>
<button type="button" id="myBtn">Click Me</button>
<script src="js/hello.js"></script>
</body>
</html>
```

Placing the JavaScript Code Inline

You can also place JavaScript code inline by inserting it directly inside the HTML tag using the special tag attributes such as `onclick`, `onmouseover`, `onkeypress`, `onload`, etc.

However, you should avoid placing large amount of JavaScript code inline as it clutters up your HTML with JavaScript and makes your JavaScript code difficult to maintain. Here's an example:

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"> <title>In lining JavaScript</title> </head>
<body><button onclick="alert('Hello World!')">Click Me</button>
</body>
</html>
```

Positioning of Script inside HTML Document

The `<script>` element can be placed in the `<head>`, or `<body>` section of an HTML document. But ideally, scripts should be placed at the end of the body section, just before the closing `</body>` tag, it will make your web pages load faster, since it prevents obstruction of initial page rendering.

Each `<script>` tag blocks the page rendering process until it has fully downloaded and executed the JavaScript code, so placing them in the head section (i.e. `<head>` element) of the document without any valid reason will significantly impact your website performance.

Difference Between Client-side and Server-side Scripting

Client-side scripting languages such as JavaScript, VBScript, etc. are interpreted and executed by the web browser, while server-side scripting languages such as PHP, ASP, Java, Python, Ruby, etc. runs on the web server and the output sent back to the web browser in HTML format.

Client-side scripting has many advantages over traditional server-side scripting approach. For example, you can use JavaScript to check if the user has entered invalid data in form fields and show notifications for input errors accordingly in real-time before submitting the form to the web-server for final data validation and further processing in order to prevent unnecessary network bandwidth usages and the exploitation of server system resources.

Also, response from a server-side script is slower as compared to a client-side script, because server-side scripts are processed on the remote computer not on the user's local computer.

JavaScript Syntax

Understanding the JavaScript Syntax

The syntax of JavaScript is the set of rules that define a correctly structured JavaScript program. A JavaScript consists of JavaScript statements that are placed within the `<script></script>` HTML tags in a web page, or within the external JavaScript file having `.js` extension. The following example shows how JavaScript statements look like:

```
var x = 5;
var y = 10;
var sum = x + y;
document.write(sum); // Prints variable value
```

Case Sensitivity in JavaScript: JavaScript is case-sensitive. This means that variables, language keywords, function names, and other identifiers must always be typed with a consistent capitalization of letters. For

example, the variable `myVar` must be typed `myVar` not `MyVar` or `myvar`. Similarly, the method name `getElementById()` must be typed with the exact case not as `getElementByID()`.

JavaScript Comments

A comment is simply a line of text that is completely ignored by the JavaScript interpreter. Comments are usually added with the purpose of providing extra information pertaining to source code. It will not only help you understand your code when you look after a period of time but also others who are working with you on the same project. JavaScript support single-line as well as multi-line comments.

Single-line comments begin with a double forward slash (`//`), followed by the comment text. Here's an example:

```
// This is my first JavaScript program
document.write("Hello World!");
```

Whereas, a multi-line comment begins with a slash and an asterisk (`/*`) and ends with an asterisk and slash (`*/`). Here's an example of a multi-line comment.

```
/* This is my first program in JavaScript */
```

```
document.write("Hello World!");
```

JavaScript Variables

What is Variable?

Variables are fundamental to all programming languages. Variables are used to store data, like string of text, numbers, etc. The data or value stored in the variables can be set, updated, and retrieved whenever needed. In general, variables are symbolic names for values. You can create a variable with the `var` keyword, whereas the assignment operator (`=`) is used to assign value to a variable, like this: `varName = value;`

```
var name = "Peter";
```

```
var age = 26;
```

```
var isMarried = false;
```

Tip: Always give meaningful names to your variables. Additionally, for naming the variables that contain multiple words, camelCase is commonly used. In this convention all word after the first should have uppercase first letters, e.g. `myLongVariableName`.

In the above example we have created three variables, first one has assigned with a string value, the second one has assigned with a number, whereas the last one assigned with a boolean value. Variables can hold different types of data. In JavaScript, variables can also be declared without having any initial values assigned to them. This is useful for variables which are supposed to hold values like user inputs.

```
// Declaring Variable
```

```
var userName;
```

```
// Assigning value
```

```
userName = "Clark Kent";
```

Declaring Multiple Variables at Once: In addition, you can also declare multiple variables and set their initial values in a single statement. Each variable is separated by commas, as demonstrated in the following example:

```
// Declaring multiple Variables
```

```
var name = "Peter Parker", age = 21, isMarried = false;
```

```
/* Longer declarations can be written to span multiple lines to improve the readability */
```

```
var name = "Peter Parker",
```

```
age = 21,
```

```
isMarried = false;
```

Naming Conventions for JavaScript Variables: Rules for naming a JavaScript variable:

A variable name must start with a letter, underscore (`_`), or dollar sign (`$`).

A variable name cannot start with a number.

A variable name can only contain alpha-numeric characters (A-z, 0-9) and underscores.

A variable name cannot contain spaces.

A variable name cannot be a JavaScript keyword or a JavaScript reserved word.

Note: Variable names in JavaScript are case sensitive, it means \$myvar and \$myVar are two different variables. So be careful while defining variable names.

Generating Output in JavaScript: There are certain situations in which you may need to generate output from your JavaScript code. For example, you might want to see the value of variable, or write a message to browser console to help you debug an issue in your running JavaScript code, and so on.

In JavaScript there are several different ways of generating output including writing output to the browser window or browser console, displaying output in dialog boxes, writing output into an HTML element, etc. We'll take a closer look at each of these in the following sections.

Writing Output to Browser Console: You can easily output the message or write data to the browser console using the `console.log()` method. This is a simple, but very powerful method for generating detailed output. Here's an example:

```
// Printing a simple text message
```

```
console.log("Hello World!");
```

```
// Prints: Hello World!
```

```
// Printing a variable value
```

```
var x = 10;
```

```
var y = 20;
```

```
var sum = x + y;
```

```
console.log(sum); // Prints: 30
```

Displaying Output in Alert Dialog Boxes

You can also use alert dialog boxes to display the message or output data to the user. An alert dialog box is created using the `alert()` method. Here's is an example:

```
// Displaying a simple text message
```

```
alert("Hello World!"); // Outputs: Hello World!
```

```
// Displaying a variable value
```

```
var x = 10;
```

```
var y = 20;
```

```
var sum = x + y;
```

```
alert(sum); // Outputs: 30
```

Writing Output to the Browser Window

You can use the `document.write()` method to write the content to the current document only while that document is being parsed. Here's an example:

```
// Printing a simple text message
```

```
document.write("Hello World!"); // Prints: Hello World!
```

```
// Printing a variable value
```

```
var x = 10; var y = 20;
```

```
var sum = x + y;
```

```
document.write(sum); // Prints: 30
```

If you use the `document.write()` method after the page has been loaded, it will overwrite all the existing content in that document. Check out the following example:

```
<h1>This is a heading</h1>
```

```
<p>This is a paragraph of text.</p>
```

```
<button type="button" onclick="document.write('Hello World!')">Click Me</button>
```

Inserting Output Inside an HTML Element

You can also write or insert output inside an HTML element using the element's `innerHTML` property. However, before writing the output first we need to select the element using a method such as `getElementById()`, as demonstrated in the following example:

```
<p id="greet"></p> <p id="result"></p> <script>
// Writing text string inside an element
document.getElementById("greet").innerHTML = "Hello World!";
// Writing a variable value inside an element var x = 10; var y = 20; var sum = x + y;
document.getElementById("result").innerHTML = sum; </script>
```

Data Types in JavaScript

Data types basically specify what kind of data can be stored and manipulated within a program.

There are six basic data types in JavaScript which can be divided into three main categories: primitive (or *primary*), *composite* (or *reference*), and *special* data types. String, Number, and Boolean are primitive data types. Object, Array, and Function (which are all types of objects) are composite data types. Whereas Undefined and Null are special data types.

Primitive data types can hold only one value at a time, whereas composite data types can hold collections of values and more complex entities. Let's discuss each one of them in detail.

The String Data Type

The *string* data type is used to represent textual data (i.e. sequences of characters). Strings are created using single or double quotes surrounding one or more characters, as shown below:

```
var a = 'Hi there!'; // using single quotes
var b = "Hi there!"; // using double quotes
```

You can include quotes inside the string as long as they don't match the enclosing quotes.

```
var a = "Let's have a cup of coffee."; // single quote inside double quotes
var b = 'He said "Hello" and left.'; // double quotes inside single quotes
var c = 'We'll never give up.'; // escaping single quote with backslash
```

The Number Data Type

The *number* data type is used to represent positive or negative numbers with or without decimal place, or numbers written using exponential notation e.g. $1.5e-4$ (equivalent to 1.5×10^{-4}).

```
var a = 25; // integer
var b = 80.5; // floating-point number
var c = 4.25e+6; // exponential notation, same as 4.25e6 or 4250000
var d = 4.25e-6; // exponential notation, same as 0.00000425
```

The Number data type also includes some special values which are: Infinity, -Infinity and NaN. Infinity represents the mathematical Infinity ∞ , which is greater than any number. Infinity is the result of dividing a nonzero number by 0, as demonstrated below:

```
alert(16 / 0); // Output: Infinity
alert(-16 / 0); // Output: -Infinity
alert(16 / -0); // Output: -Infinity
```

While NaN represents a special Not-a-Number value. It is a result of an invalid or an undefined mathematical operation, like taking the square root of -1 or dividing 0 by 0, etc.

```
alert("Some text" / 2); // Output: NaN
alert("Some text" / 2 + 10); // Output: NaN
alert(Math.sqrt(-1)); // Output: NaN
```

The Boolean Data Type

The Boolean data type can hold only two values: `true` or `false`. It is typically used to store values like yes (`true`) or no (`false`), on (`true`) or off (`false`), etc. as demonstrated below:

Example

```
var isReading = true; // yes, I'm reading
var isSleeping = false; // no, I'm not sleeping
```

The Array Data Type

An array is a type of object used for storing multiple values in single variable. Each value (also called an element) in an array has a numeric position, known as its index, and it may contain data of any data type—numbers, strings, booleans, functions, objects, and even other arrays. The array index starts from 0, so that the first array element is `arr[0]` not `arr[1]`.

The simplest way to create an array is by specifying the array elements as a comma-separated list enclosed by square brackets, as shown in the example below:

```
var colors = ["Red", "Yellow", "Green", "Orange"];
var cities = ["London", "Paris", "New York"];
alert(colors[0]); // Output: Red
alert(cities[2]); // Output: New York
```

The Function Data Type

The function is callable object that executes a block of code. Since functions are objects, so it is possible to assign them to variables, as shown in the example below:

Example

```
var greeting = function(){
    return "Hello World!";
}
// Check the type of greeting variable
alert(typeof greeting) // Output: function
alert(greeting()); // Output: Hello World!
```

In fact, functions can be used at any place any other value can be used. Functions can be stored in variables, objects, and arrays. Functions can be passed as arguments to other functions, and functions can be returned from functions. Consider the following function:

```
function createGreeting(name){
    return "Hello, " + name;
}
function displayGreeting(greetingFunction, userName){
    return greetingFunction(userName);
}
var result = displayGreeting(createGreeting, "Peter"); alert(result); // Output: Hello, Peter
```

JavaScript Operators

What are Operators in JavaScript

Operators are symbols or keywords that tell the JavaScript engine to perform some sort of actions. For example, the addition (+) symbol is an operator that tells JavaScript engine to add two variables or values, while the equal-to (==), greater-than (>) or less-than (<) symbols are the operators that tells JavaScript engine to compare two variables or values, and so on. The following sections describe the different operators used in JavaScript.

JavaScript Arithmetic Operators

The arithmetic operators are used to perform common arithmetical operations, such as addition, subtraction, multiplication etc. Here's a complete list of JavaScript's arithmetic operators:

The following example will show you these arithmetic operators in action:

JavaScript Assignment Operators

The assignment operators are used to assign values to variables.

Example

```
var x; // Declaring Variable
x = 10; alert(x); // Outputs: 10
x = 20; x += 30;
alert(x); // Outputs: 50
x = 50; x -= 20;
alert(x); // Outputs: 30
x = 5; x *= 25;
alert(x); // Outputs: 125
x = 50; x /= 10;
alert(x); // Outputs: 5
x = 100; x %= 15;
alert(x); // Outputs: 10
```

JavaScript String Operators

There are two operators which can also be used for strings.

The following example will show you these string operators in action:

```
var str1 = "Hello";
var str2 = " World!";
alert(str1 + str2); // Outputs: Hello World!
str1 += str2;
alert(str1); // Outputs: Hello World!
```

JavaScript Incrementing and Decrementing Operators

The increment/decrement operators are used to increment/decrement a variable's value.

```
var x; // Declaring Variable
x = 10; alert(++x); // Outputs: 11
alert(x); // Outputs: 11 x = 10;
alert(x++); // Outputs: 10
alert(x); // Outputs: 11
x = 10; alert(--x); // Outputs: 9
alert(x); // Outputs: 9
x = 10; alert(x--); // Outputs: 10
alert(x); // Outputs: 9
```

JavaScript Logical Operators

The logical operators are typically used to combine conditional statements.

The following example will show you how these logical operators actually work:

```
var year = 2018; // Leap years are divisible by 400 or by 4 but not 100
if((year % 400 == 0) || ((year % 100 != 0) && (year % 4 == 0)))
{
    alert(year + " is a leap year.");
}
else{
    alert(year + " is not a leap year.");
}
```

JavaScript Comparison Operators

The comparison operators are used to compare two values in a Boolean fashion.

```
var x = 25;
var y = 35;
var z = "25";
alert(x == z); // Outputs: true
alert(x === z); // Outputs: false
alert(x != y); // Outputs: true
alert(x !== z); // Outputs: true
alert(x < y); // Outputs: true
alert(x > y); // Outputs: false
alert(x <= y); // Outputs: true
alert(x >= y); // Outputs: false
```

JavaScript Functions

What is Function?

A function is a group of statements that perform specific tasks and can be kept and maintained separately from main program. Functions provide a way to create reusable code packages which are more portable and easier to debug. Here are some advantages of using functions:

A function reduces the repetition of code within a program: Function allows you to extract commonly used block of code into a single component. Now you can perform the same task by calling this function wherever you want within your script without having to copy and paste the same block of code again and again.

A function makes the code much easier to maintain: Since a function created once can be used many times, so any changes made inside a function automatically implemented at all the places without touching the several files.

Functions makes it easier to eliminate the errors: When the program is subdivided into functions, if any error occur you know exactly what function causing the error and where to find it. Therefore, fixing errors becomes much easier. The following section will show you how to define and call functions in your scripts.

Defining and Calling a Function: The declaration of a function start with the `function` keyword, followed by the name of the function you want to create, followed by parentheses i.e. `()` and finally place your function's code between curly brackets `{ }`. Here's the basic syntax for declaring a function:

```
function functionName() {
    // Code to be executed
}
```

Here is a simple example of a function, that will show a hello message:

Example

```
// Defining function
function sayHello() {
    alert("Hello, welcome to this website!");
}

// Calling function
sayHello(); // Outputs: Hello, welcome to this website!
```

Once a function is defined it can be called (invoked) from anywhere in the document, by typing its name followed by a set of parentheses, like `sayHello()` in the example above.

Note: A function name must start with a letter or underscore character not with a number, optionally followed by the more letters, numbers, or underscore characters. Function names are case sensitive, just like variable names.

Adding Parameters to Functions

You can specify parameters when you define your function to accept input values at run time. The parameters work like placeholder variables within a function; they're replaced at run time by the values (known as argument) provided to the function at the time of invocation.

Parameters are set on the first line of the function inside the set of parentheses, like this:

```
function functionName(parameter1, parameter2, parameter3) {  
    // Code to be executed  
}
```

The `displaySum()` function in the following example takes two numbers as arguments, simply add them together and then display the result in the browser.

Example

// Defining function

```
function displaySum(num1, num2) {  
    var total = num1 + num2;  
    alert(total);  
}
```

// Calling function

```
displaySum(6, 20); // Outputs: 26
```

```
displaySum(-5, 17); // Outputs: 12
```

You can define as many parameters as you like. However for each parameter you specify, a corresponding argument needs to be passed to the function when it is called, otherwise its value becomes `undefined`. Let's consider the following example:

Example

// Defining function

```
function showFullname(firstName, lastName) {  
    alert(firstName + " " + lastName);  
} // Calling function
```

```
showFullname("Clark", "Kent"); // Outputs: Clark Kent
```

```
showFullname("John"); // Outputs: John undefined
```

Returning Values from a Function: A function can return a value back to the script that called the function as a result using the `return` statement. The value may be of any type, including arrays and objects. The `return` statement usually placed as the last line of the function before the closing curly bracket and ends it with a semicolon, as shown in the following example.

Example

// Defining function

```
function getSum(num1, num2) {  
    var total = num1 + num2;  
    return total;  
}
```

// Displaying returned value

```
alert(getSum(6, 20)); // Outputs: 26
```

```
alert(getSum(-5, 17)); // Outputs: 12
```

A function *cannot* return multiple values. However, you can obtain similar results by returning an array of values, as demonstrated in the following example.

Example

```
// Defining function
function divideNumbers(dividend, divisor){
    var quotient = dividend / divisor;
    var arr = [dividend, divisor, quotient];
    return arr;
}
// Store returned value in a variable
var all = divideNumbers(10, 2);
// Displaying individual values
alert(all[0]); // Outputs: 10
alert(all[1]); // Outputs: 2
alert(all[2]); // Outputs: 5
```

Working with Function Expressions: The syntax that we've used before to create functions is called function declaration. There is another syntax for creating a function that is called a function expression.

Example

```
// Function Declaration
function getSum(num1, num2) {
    var total = num1 + num2;
    return total;
}
// Function Expression
var getSum = function(num1, num2) {
    var total = num1 + num2;
    return total;
};
```

Once function expression has been stored in a variable, the variable can be used as a function:

Example

```
var getSum = function(num1, num2) {
    var total = num1 + num2;
    return total;
};
alert(getSum(5, 10)); // Outputs: 15
var sum = getSum(7, 25);
alert(sum); // Outputs: 32
```

Note: There is no need to put a semicolon after the closing curly bracket in a function declaration. But function expressions, on the other hand, should always end with a semicolon.

Tip: In JavaScript functions can be stored in variables, passed into other functions as arguments, passed out of functions as return values, and constructed at run-time.

The syntax of the function declaration and function expression looks very similar, but they differ in the way they are evaluated, check out the following example:

Example

```
declaration(); // Outputs: Hi, I'm a function declaration!
function declaration() {
    alert("Hi, I'm a function declaration!");
}
expression(); // Uncaught TypeError: undefined is not a function
var expression = function() {
    alert("Hi, I'm a function expression!");
};
```

```
};
```

As you can see in the above example, the function expression threw an exception when it was invoked before it is defined, but the function declaration executed successfully.

JavaScript parse declaration function before the program executes. The function expression is not evaluated until it is assigned to a variable; therefore, it is still undefined when invoked.

Understanding the Variable Scope: However, you can declare the variables anywhere in JavaScript. But, the location of the declaration determines the extent of a variable's availability within the JavaScript program i.e. where the variable can be used or accessed. This accessibility is known as *variable scope*.

By default, variables declared within a function have *local scope* that means they cannot be viewed or manipulated from outside of that function, as shown in the example below:

Example

```
// Defining function
```

```
function greetWorld() {  
    var greet = "Hello World!";  
    alert(greet);  
}
```

```
greetWorld(); // Outputs: Hello World!
```

```
alert(greet); // Uncaught ReferenceError: greet is not defined
```

However, any variables declared in a program outside of a function has *global scope* i.e. it will be available to all script, whether that script is inside a function or outside. Here's an example:

Example

```
var greet = "Hello World!";
```

```
// Defining function
```

```
function greetWorld() {  
    alert(greet);  
}
```

```
greetWorld(); // Outputs: Hello World!
```

```
alert(greet); // Outputs: Hello World!
```

JavaScript If...Else Statements

JavaScript Conditional Statements

Like many other programming languages, JavaScript also allows you to write code that perform different actions based on the results of a logical or comparative test conditions at run time. This means, you can create test conditions in the form of expressions that evaluates to either true or false and based on these results you can perform certain actions.

There are several conditional statements in JavaScript that you can use to make decisions:

The **if** statement

The **if...else** statement

The **if...else if...else** statement

The **switch...case** statement

We will discuss each of these statements in detail in the coming sections.

The if Statement

The *if* statement is used to execute a block of code only if the specified condition evaluates to true. This is the simplest JavaScript's conditional statements and can be written like:

```
if(condition) {  
    // Code to be executed  
}
```

The following example will output "Have a nice weekend!" if the current day is Friday:

Example

```

var now = new Date();
var dayOfWeek = now.getDay(); // Sunday - Saturday : 0 - 6
if(dayOfWeek == 5) {
    alert("Have a nice weekend!");
}

```

The if...else Statement

You can enhance the decision making capabilities of your JavaScript program by providing an alternative choice through adding an *else* statement to the *if* statement.

The *if...else* statement allows you to execute one block of code if the specified condition is evaluating to true and another block of code if it is evaluating to false. It can be written, like this:

```

if(condition) {
    // Code to be executed if condition is true
} else {
    // Code to be executed if condition is false
}

```

The JavaScript code in the following example will output "Have a nice weekend!" if the current day is Friday, otherwise it will output the text "Have a nice day!".

Example

```

var now = new Date();
var dayOfWeek = now.getDay(); // Sunday - Saturday : 0 - 6
if(dayOfWeek == 5) {
    alert("Have a nice weekend!");
} else {
    alert("Have a nice day!");
}

```

The if...else if...else Statement

The *if...else if...else* a special statement that is used to combine multiple *if...else* statements.

```

if(condition1) {
    // Code to be executed if condition1 is true
} else if(condition2) {
    // Code to be executed if the condition1 is false and condition2 is true
} else {
    // Code to be executed if both condition1 and condition2 are false
}

```

The following example will output "Have a nice weekend!" if the current day is Friday, and "Have a nice Sunday!" if the current day is Sunday, otherwise it will output "Have a nice day!"

Example

```

var now = new Date();
var dayOfWeek = now.getDay(); // Sunday - Saturday : 0 - 6
if(dayOfWeek == 5) {
    alert("Have a nice weekend!");
} else if(dayOfWeek == 0) {
    alert("Have a nice Sunday!");
} else {
    alert("Have a nice day!");
}

```

The Ternary Operator: The ternary operator provides a shorthand way of writing the *if...else* statements. The ternary operator is represented by the question mark (?) symbol and it takes three operands: a condition to check, a result for true, and a result for false. Its basic syntax is:

```
var result = (condition) ? value1 : value2
```

If the condition is evaluated to true the value1 will be returned, otherwise value2 will be returned. To understand how this operator works, consider the following examples:

Example

```
var userType;
var age = 21;
if(age < 18) {
    userType = 'Child';
} else {
    userType = 'Adult';
}
alert(userType); // Displays Adult
```

Using the ternary operator, the same code could be written in a more compact way:

Example

```
var age = 21;
var userType = age < 18 ? 'Child' : 'Adult';
alert(userType); // Displays Adult
```

As you can see in the above example, since the specified condition evaluated to false the value on the right side of the colon (:) is returned, which is the string 'Adult'.

Tip: Code written using the ternary operator can be difficult to read sometimes. However, it provides a great way to write compact if-else statements.

JavaScript: Form Validation

Validating form input with JavaScript is easy to do and can save a lot of unnecessary calls to the server. It can prevent people from leaving fields blank, from entering too little or too much or from using invalid characters.

User-submitted data should also *always* be verified using a secure server-side script. Otherwise a browser with JavaScript disabled, or a hacker trying to compromise your site, can easily by-pass client-side validation.

1. Restricting input to alphanumeric characters

In the following example, the single input box, **inputfield**, must:

- a) not be empty; and
- b) contain only alphanumeric characters and spaces. Only if both tests are passed can the form be submitted (when the script returns a value of true).

```
<script>
function checkForm(form)
{
    // validation fails if the input is blank
    if(form.inputfield.value == "") {
        alert("Error: Input is empty!");
        form.inputfield.focus();
        return false;
    }
    // regular expression to match only alphanumeric characters and spaces
    var re = /^[ \w ]+$/;
    // validation fails if the input doesn't match our regular expression
```



```

    if(!re.test(form.inputfield.value)) {
        alert("Error: Input contains invalid characters!");
        form.inputfield.focus();
        return false;
    }
    // validation was successful
    return true;
}
</script>

```

The form HTML is put together as follows:

```

<form method="POST" action="form-handler" onsubmit="return checkForm(this);">
<p>Input: <input type="text" size="32" name="inputfield">
<input type="submit"></p>
</form>

```

The name attribute of the input field is used to reference that field from within the checkForm function. When the form is submitted - either by hitting Enter or clicking on the Submit button - the onsubmit handler is triggered.

This then calls our checkForm() function, passing a reference to itself (the form) as the only variable. This makes the value of the input box available within the function as form.inputfield.value (the value of the field called 'inputfield' belonging to the form).

In special cases, when the name is a variable or contains special characters, you might also see form.elements[inputfield].value.

Other form values are available using a similar syntax, although this becomes more complicated if you're using SELECT lists, checkboxes or radio buttons (see below for examples).

The checkForm function tests the form input against our conditions, returning a value of true if the form is to be submitted (when all tests have been passed) or false to abort (cancel) the form submission. It's that simple.

2. Validating Text Input Fields

The value of a text input box (or a textarea or password input) is available using the syntax form.fieldname.value. This is not the case for other input types.

form.fieldname.value

To check whether two inputs have the same is quite simple:

```

if(form.field1.value == form.field2.value) {
    // values are identical
}

```

Make sure to always use `==` for comparisons. If you use `=` (the assignment operator) instead then it can take a long time to debug.

and to see if they have different values we just reverse the logic:

```

if(form.field1.value != form.field2.value) {
    // values are different
}

```

If you want to test numeric values (or add or subtract them) then you first have to convert them from strings to numbers. By default, all form values are accessed as strings.

```

var field1 = parseInt(form.field1.value);
var field2 = parseInt(form.field2.value);
if(field1 > field2) {
    // field1 as a number is greater than field2 as a number
}

```

```
}
```

parseFloat is the same as parseInt except that it also works for floating point numbers.

3. Validating Drop-down Lists

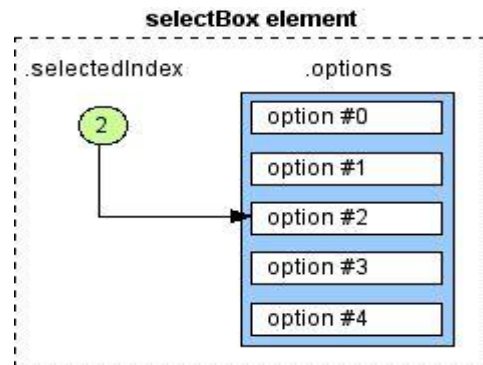
The value of a SELECT input element is accessed using:

```
var selectBox = form.fieldname;
```

```
var selectedValue = selectBox.options[selectBox.selectedIndex].value
```

```
var selectedText = selectBox.options[selectBox.selectedIndex].text
```

where `fieldname` is the SELECT element, which has an array of `options` and a value `selectedIndex` that tells you which option has been selected. The illustration below shows this relationship:



Note:

'I' in `selectedIndex` needs to be capitalised - JavaScript functions and variables are **always** case-sensitive.

If you define a `value` for the OPTION elements in your SELECT list, then `.value` will return that, while `.text` will return the text that is visible in the browser. Here's an example of what this refers to:

```
<option value="value">text</option>
```

4. Validating Checkbox: These really are simple:

```
form.checkboxfield.checked
```

will return a boolean value (true or false) indicating whether the checkbox is in a 'checked' state.

```
function checkForm(form)
{
  if(form.checkboxfield.checked) {
    alert("The checkbox IS checked");
  } else {
    alert("The checkbox IS NOT checked");
  }
  return false;
}
```

You don't need to test using `form.checkboxfield.checked == true` as the value is already boolean.

5. Validating Radio buttons: Radio buttons are implemented as if they were an array of checkboxes. To find out which value (if any) has been selected, you need to loop through the array until you find which one has been selected:

```
<script>
function checkRadio(field){
  for(var i=0; i < field.length; i++) {
    if(field[i].checked) return field[i].value;
  } return false;
}
</script>
```

The form handler function is then the following:

```
function checkForm(form)
```

```
{  
  if (radioValue = checkRadio(form.radiofield)) {  
    alert("You selected " + radioValue);  
    return true;  
  } else {  
    alert("Error: No value was selected!");  
    return false;  
  }  
}
```

WEB PROGRAMMING MODULE

CHAPTER FOUR

HYPERTEXT PREPROCESSOR (PHP)

PHP - Introduction

PHP is a recursive acronym for "PHP: Hypertext Preprocessor".

PHP is a server side scripting language that is embedded in HTML. It is used to manage dynamic content, databases, session tracking, even build entire e-commerce sites.

It is integrated with a number of popular databases, including MySQL, PostgreSQL, Oracle, Sybase, Informix, and Microsoft SQL Server.

PHP is pleasingly zippy in its execution, especially when compiled as an Apache module on the Unix side. The MySQL server, once started, executes even very complex queries with huge result sets in record-setting time.

PHP supports a large number of major protocols such as POP3, IMAP, and LDAP. PHP4 added support for Java and distributed object architectures (COM and CORBA), making n-tier development a possibility for the first time.

PHP is forgiving: PHP language tries to be as forgiving as possible.

PHP Syntax is C-Like.

Common uses of PHP

PHP performs system functions, i.e. from files on a system it can create, open, read, write, and close them. PHP can handle forms, i.e. gather data from files, save data to a file, through email you can send data, return data to the user.

You add, delete, modify elements within your database through PHP.

Access cookies variables and set cookies.

Using PHP, you can restrict users to access some pages of your website.

It can encrypt data.

Characteristics of PHP

Five important characteristics make PHP's practical nature possible –

Simplicity

Efficiency

Security

Flexibility

Familiarity

"Hello World" Script in PHP

To get a feel for PHP, first start with simple PHP scripts. Since "Hello, World!" is an essential example, first we will create a friendly little "Hello, World!" script.

As mentioned earlier, PHP is embedded in HTML. That means that in amongst your normal HTML (or XHTML if you're cutting-edge) you'll have PHP statements like this:

```
<html>
<head>
  <title>Hello World</title>
</head>
<body>
  <?php echo "Hello, World!";?>
</body>
</html>
```

It will produce: Hello, World!

If you examine the HTML output of the above example, you'll notice that the PHP code is not present in the file sent from the server to your Web browser. All of the PHP present in the Web page is processed

and stripped from the page; the only thing returned to the client from the Web server is pure HTML output.

All PHP code must be included inside one of the three special markup tags ATE are recognized by the PHP Parser.

```
<?php PHP code goes here ?>
<? PHP code goes here ?>
<script language = "php"> PHP code goes here </script>
```

A most common tag is the `<?php...?>` and we will also use the same tag in our tutorial.

From the next chapter we will start with PHP Environment Setup on your machine and then we will dig out almost all concepts related to PHP to make you comfortable with the PHP language.

PHP - Environment Setup

In order to develop and run PHP Web pages three vital components need to be installed on your computer system.

Web Server: PHP will work with virtually all Web Server software, including Microsoft's Internet Information Server (IIS) but then most often used is freely available Apache Server.

Database: PHP will work with virtually all database software, including Oracle and Sybase but most commonly used is freely available MySQL database.

PHP Parser: In order to process PHP script instructions a parser must be installed to generate HTML output that can be sent to the Web Browser.

PHP Parser Installation

Before you proceed it is important to make sure that you have proper environment setup on your machine to develop your web programs using PHP. Type the following address into your browser's address box.

<http://127.0.0.1/info.php>

If this displays a page showing your PHP installation related information, then it means you have PHP and Webserver installed properly. Otherwise you have to follow given procedure to install PHP on your computer.

Apache Configuration: If you are using Apache as a Web Server then this section will guide you to edit Apache Configuration Files.

PHP.INI File Configuration: The PHP configuration file, `php.ini`, is the final and most immediate way to affect PHP's functionality.

PHP - Syntax Overview: This chapter will give you an idea of very basic syntax of PHP and very important to make your PHP foundation strong.

Escaping to PHP: The PHP parsing engine needs a way to differentiate PHP code from other elements in the page. The mechanism for doing so is known as 'escaping to PHP'. There are four ways to do this –

Canonical PHP tags

The most universally effective PHP tag style is –

```
<?php...?>
```

If you use this style, you can be positive that your tags will always be correctly interpreted.

Short-open (SGML-style) tags

Short or short-open tags look like this –

```
<?...?>
```

Short tags are, as one might expect, the shortest option You must do one of two things to enable PHP to recognize the tags –

Choose the `--enable-short-tags` configuration option when you're building PHP.

Set the `short_open_tag` setting in your `php.ini` file to on. This option must be disabled to parse XML with PHP because the same syntax is used for XML tags.

ASP-style tags

ASP-style tags mimic the tags used by Active Server Pages to delineate code blocks. ASP-style tags look like this –

```
<%...%>
```

To use ASP-style tags, you will need to set the configuration option in your php.ini file.

HTML script tags

HTML script tags look like `<script language = "PHP">...</script>`

Commenting PHP Code: A comment is the portion of a program that exists only for the human reader and stripped out before displaying the programs result. There are two commenting formats in PHP.

Single-line comments – They are generally used for short explanations or notes relevant to the local code. Here are the examples of single line comments.

```
<?
```

```
# This is a comment, and
# This is the second line of the comment
// This is a comment too. Each style comments only
print "An example with single line comments";
```

```
?>
```

Multi-lines comments: They are generally used to provide **pseudocode** algorithms and more detailed explanations when necessary. The multiline style of commenting is the same as in C. Here is an example of multi lines comments.

```
<?
```

```
/* This is a comment with multiline
   Author: Lecturer Abebe Alambo
   Purpose: Multiline Comments
   Subject: PHP
*/
print "An example with multi line comments";
```

```
?>
```

PHP is whitespace insensitive

Whitespace is the stuff you type that is typically invisible on the screen, including spaces, tabs, and carriage returns (end-of-line characters).

PHP whitespace insensitive means that it almost never matters how many whitespace characters you have in a row. one whitespace character is the same as many such characters.

For example, each of the following PHP statements that assigns the sum of 2 + 2 to the variable \$four is equivalent –

```
$four = 2 + 2; // single spaces
$four <tab>=<tab2<tab>+<tab>2 ; // spaces and tabs
$four =2+2; // multiple lines
```

PHP is case sensitive

Yeah it is true that PHP is a case sensitive language. Try out following example –

```
<html><body>
    <?php
    $capital = 67;
    print("Variable capital is $capital<br>");
    print("Variable CaPiTaL is $CaPiTaL<br>");
    ?></body>
</html>
```

This will produce the following result – Variable capital is 67

Variable CaPiTaL is

Statements are expressions terminated by semicolons

A statement in PHP is any expression that is followed by a semicolon (;). Any sequence of valid PHP statements that is enclosed by the PHP tags is a valid PHP program. Here is a typical statement in PHP, which in this case assigns a string of characters to a variable called \$greeting.

```
$greeting = "Welcome to PHP!";
```

Expressions are combinations of tokens

The smallest building blocks of PHP are the indivisible tokens, such as numbers (3.14159), strings (.two.), variables (\$two), constants (TRUE), and the special words that make up the syntax of PHP itself like if, else, while, for and so forth.

Braces make blocks

Although statements cannot be combined like expressions, you can always put a sequence of statements anywhere a statement can go by enclosing them in a set of curly braces.

Here both statements are equivalent:

```
if (3 == 2 + 1)
    print("Good - I haven't totally lost my mind.<br>");
if (3 == 2 + 1) {
    print("Good - I haven't totally");
    print("lost my mind.<br>");
}
```

PHP - Variable Types

The main way to store information in the middle of a PHP program is by using a variable.

Here are the most important things to know about variables in PHP.

All variables in PHP are denoted with a leading dollar sign (\$).

The value of a variable is the value of its most recent assignment.

Variables are assigned with the = operator, with the variable on the left-hand side and the expression to be evaluated on the right.

Variables can, but do not need, to be declared before assignment.

Variables in PHP do not have intrinsic types - a variable does not know in advance whether it will be used to store a number or a string of characters.

Variables used before they are assigned have default values.

PHP does a good job of automatically converting types from one to another when necessary.

PHP variables are Perl-like.

PHP has a total of eight data types which we use to construct our variables

Integers – are whole numbers, without a decimal point, like 4195.

Doubles – are floating-point numbers, like 3.14159 or 49.1.

Booleans – have only two possible values either true or false.

NULL – is a special type that only has one value: NULL.

Strings – are sequences of characters, like 'PHP supports string operations.'

Arrays – are named and indexed collections of other values.

Objects – are instances of programmer-defined classes, which can package up both other kinds of values and functions that are specific to the class.

Resources – are special variables that hold references to resources external to PHP (such as database connections).

The first five are simple types, and the next two (arrays and objects) are compound - the compound types can package up other arbitrary values of arbitrary type, whereas the simple types cannot. We will explain only simple data type in this chapters. Array and Objects will be explained separately.

Integers: They are whole numbers, without a decimal point, like 4195. They are the simplest type. they correspond to simple whole numbers, both positive and negative. Integers can be assigned to variables, or they can be used in expressions, like so –

```
$int_var = 12345;
```

```
$another_int = -12345 + 12345;
```

Integer can be in decimal (base 10), octal (base 8), and hexadecimal (base 16) format. Decimal format is the default, octal integers are specified with a leading 0, and hexadecimal have a leading 0x.

For most common platforms, the largest integer is $(2^{31} - 1)$ (or 2,147,483,647), and the smallest (most negative) integer is $-(2^{31} - 1)$ (or -2,147,483,647).

Doubles

They like 3.14159 or 49.1. By default, doubles print with the minimum number of decimal places needed.

For example, the code –

```
<?php
    $many = 2.2888800;
    $many_2 = 2.2111200;
    $few = $many + $many_2;
    print("$many + $many_2 = $few <br>");
?>
```

It produces the following browser output –

2.28888 + 2.21112 = 4.5

Boolean

They have only two possible values either true or false. PHP provides a couple of constants especially for use as Booleans: TRUE and FALSE, which can be used like so –

```
if (TRUE)
    print("This will always print<br>");
else
    print("This will never print<br>");
```

Interpreting other types as Booleans

Here are the rules for determine the "truth" of any value not already of the Boolean type –

If the value is a number, it is false if exactly equal to zero and true otherwise.

If the value is a string, it is false if the string is empty (has zero characters) or is the string "0", and is true otherwise.

Values of type NULL are always false.

If the value is an array, it is false if it contains no other values, and it is true otherwise. For an object, containing a value means having a member variable that has been assigned a value.

Valid resources are true (although some functions that return resources when they are successful will return FALSE when unsuccessful).

Don't use double as Booleans.

Each of the following variables has the truth value embedded in its name when it is used in a Boolean context.

```
$true_num = 3 + 0.14159;
>true_str = "Tried and true"
>true_array[49] = "An array element";
>false_array = array();
>false_null = NULL;
>false_num = 999 - 999;
>false_str = "";
```


NULL: NULL is a special type that only has one value: NULL. To give a variable the NULL value, simply assign it like this –

```
$my_var = NULL;
```

The special constant NULL is capitalized by convention, but actually it is case insensitive; you could just as well have typed –

```
$my_var = null;
```

A variable that has been assigned NULL has the following properties –

It evaluates to FALSE in a Boolean context.

It returns FALSE when tested with IsSet() function.

Strings: They are sequences of characters, like "PHP supports string operations". Following are valid examples of string

```
$string_1 = "This is a string in double quotes";
```

```
$string_2 = 'This is a somewhat longer, singly quoted string';
```

```
$string_39 = "This string has thirty-nine characters";
```

```
$string_0 = ""; // a string with zero characters
```

Singly quoted strings are treated almost literally, whereas doubly quoted strings replace variables with their values as well as specially interpreting certain character sequences.

```
<?php
    $variable = "name";
    $literally = 'My $variable will not print!';
    print($literally);
    print "<br>";
    $literally = "My $variable will print!";
    print($literally);
?>
```

This will produce following result –

My \$variable will not print!

My name will print

There are no artificial limits on string length - within the bounds of available memory, you ought to be able to make arbitrarily long strings.

Strings that are delimited by double quotes (as in "this") are preprocessed in both the following two ways by PHP –

Certain character sequences beginning with backslash (\) are replaced with special characters

Variable names (starting with \$) are replaced with string representations of their values.

The escape-sequence replacements are –

\n is replaced by the newline character

\t is replaced by the tab character

\\$ is replaced by the dollar sign itself (\$)

\\" is replaced by a single double-quote (")

\\ is replaced by a single backslash (\)

Variable Scope

Scope can be defined as the range of availability a variable has to the program in which it is declared. PHP variables can be one of four scope types –

Local variables

Function parameters

Global variables

Static variables

Variable Naming

Rules for naming a variable is –

Variable names must begin with a letter or underscore character.

A variable name can consist of numbers, letters, underscores but you cannot use characters like + , - , % , (,) . & , etc

There is no size limit for variables.

PHP - Operator Types

What is Operator? Simple answer can be given using expression 4 + 5 is equal to 9. Here 4 and 5 are called operands and + is called operator. PHP language supports following type of operators.

Arithmetic Operators

Comparison Operators

Logical (or Relational) Operators

Assignment Operators

Conditional (or ternary) Operators

Let's have a look on all operators one by one.

Arithmetic Operators: There are following arithmetic operators supported by PHP language. Assume variable A holds 10 and variable B holds 20 then

Operator	Description	Example
+	Adds two operands	A + B will give 30
-	Subtracts second operand from the first	A - B will give -10
*	Multiply both operands	A * B will give 200
/	Divide numerator by de-numerator	B / A will give 2
%	Modulus Operator and remainder of after an integer division	B % A will give 0
++	Increment operator, increases integer value by one	A++ will give 11
--	Decrement operator, decreases integer value by one	A-- will give 9

Comparison Operators: There are following comparison operators supported by PHP language. Assume variable A holds 10 and variable B holds 20 then

Operator	Description	Example
==	Checks if the value of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!=	Checks if the value of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the	(A >= B) is

	value of right operand, if yes then condition becomes true.	not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

Logical Operators: There are following logical operators supported by PHP language. Assume variable A holds 10 and variable B holds 20 then

Operator	Description	Example
and	Called Logical AND operator. If both the operands are true then condition becomes true.	(A and B) is true.
or	Called Logical OR Operator. If any of the two operands are non-zero then condition becomes true.	(A or B) is true.
&&	Called Logical AND operator. If both the operands are non-zero then condition becomes true.	(A && B) is true.
	Called Logical OR Operator. If any of the two operands are non-zero then condition becomes true.	(A B) is true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is false.

Assignment Operators: There are following assignment operators supported by PHP language –

Operator	Description	Example
=	Simple assignment operator, Assigns values from right side operands to left side operand	C = A + B will assign value of A + B into C
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand	C %= A is equivalent to C = C % A

Conditional Operator: There is one more operator called conditional operator. This first evaluates an expression for a true or false value and then execute one of the two given statements depending upon the result of the evaluation. The conditional operator has this syntax.

Operator	Description	Example
? :	Conditional Expression	If Condition is true? Then value X : Otherwise value Y

Operators Categories

All the operators we have discussed above can be categorized into following categories –

Unary prefix operators, which precede a single operand.

Binary operators, which take two operands and perform a variety of arithmetic and logical operations.

The conditional operator (a ternary operator), which takes three operands and evaluates either the second or third expression, depending on the evaluation of the first expression.

Assignment operators, which assign a value to a variable.

Precedence of PHP Operators

Operator precedence determines the grouping of terms in an expression. This affects how an expression is evaluated. Certain operators have higher precedence than others; for example, the multiplication operator has higher precedence than the addition operator –

For example, $x = 7 + 3 * 2$; Here x is assigned 13, not 20 because operator * has higher precedence than + so it first gets multiplied with $3*2$ and then adds into 7.

Here operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

Category	Operator	Associativity
Unary	! ++ --	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=	Right to left

PHP - Decision Making: The if, else if ...else and switch statements are used to take decision based on the different condition. You can use conditional statements in your code to make your decisions. PHP supports following three decision making statements: -

if...else statement: use this statement if you want to execute a set of code when a condition is true and another if the condition is not true

else if statement: is used with the if...else statement to execute a set of code if **one** of the several condition is true

switch statement: is used if you want to select one of many blocks of code to be executed, use the Switch statement. The switch statement is used to avoid long blocks of if..elseif..else code.

The If...Else Statement: If you want to execute some code if a condition is true and another code if a condition is false, use the if....else statement.

Syntax

if (condition)

code to be executed if condition is true;

else

code to be executed if condition is false;

Example

The following example will output "Have a nice weekend!" if the current day is Friday, Otherwise, it will output "Have a nice day!":

```
<html>
<body>
  <?php
    $d = date("D");
    if ($d == "Fri")
      echo "Have a nice weekend!";
    else
      echo "Have a nice day!";
  ?>
</body>
</html>
```

It will produce: Have a nice weekend!

The Else If Statement

If you want to execute some code if one of the several conditions are true use the elseif statement

Syntax

if (condition)

code to be executed if condition is true;

elseif (condition)

code to be executed if condition is true;

else

code to be executed if condition is false;

Example

The following example will output "Have a nice weekend!" if the current day is Friday, and "Have a nice Sunday!" if the current day is Sunday. Otherwise, it will output "Have a nice day!" –

```
<html><body>
  <?php
    $d = date("D");
    if ($d == "Fri")
      echo "Have a nice weekend!";
    elseif ($d == "Sun")
      echo "Have a nice Sunday!";
    else
      echo "Have a nice day!";
  ?>
</body>
</html>
```

It will produce the following result –

Have a nice Weekend!

The Switch Statement

If you want to select one of many blocks of code to be executed, use the Switch statement.

The switch statement is used to avoid long blocks of if..elseif..else code.

Syntax

```
switch (expression){
    case label1:
        code to be executed if expression = label1;
        break;
    case label2:
        code to be executed if expression = label2;
        break;
    default:
        code to be executed
        if expression is different
        from both label1 and label2;
}
```

PHP - Loop Types: Loops in PHP are used to execute the same block of code a specified number of times. PHP supports following four loop types.

for : loops through a block of code a specified number of times.

While: loops through a block of code if and as long as a specified condition is true.

do...while: loops through a block of code once, and then repeats the loop as long as a special condition is true.

Foreach: loops through a block of code for each element in an array.

We will discuss about **continue** and **break** keywords used to control the loops execution.

The for loop statement

The for statement is used when you know how many times you want to execute a statement or a block of statements.

Syntax

```
for (initialization; condition; increment){
    code to be executed;
}
```

The initializer is used to set the start value for the counter of the number of loop iterations. A variable may be declared here for this purpose and it is traditional to name it \$i.

Example

The following example makes five iterations and changes the assigned value of two variables on each pass of the loop –

```
<html><body><?php
    $a = 0;
    $b = 0;
    for( $i = 0; $i<5; $i++ ) {
        $a += 10;
        $b += 5;
    }
    echo ("At the end of the loop a = $a and b = $b" );
?>
</body></html>
```

This will produce: At the end of the loop a = 50 and b = 25

The while loop statement: The while statement will execute a block of code if and as long as a test expression is true. If the test expression is true then the code block will be executed. After the code has executed the test expression will again be evaluated and the loop will continue until the test expression is found to be false.

Syntax

```
while (condition) {  
    code to be executed;  
}
```

Example

This example decrements a variable value on each iteration of the loop and the counter increments until it reaches 10 when the evaluation is false and the loop ends.

```
<html>  
<body>  
    <?php  
        $i = 0;  
        $num = 50;  
        while( $i < 10) {  
            $num--;  
            $i++;  
        }  
        echo ("Loop stopped at i = $i and num = $num" );  
    ?>  
</body>  
</html>
```

This will produce: Loop stopped at i = 10 and num = 40

The do...while loop statement

The do...while statement will execute a block of code at least once - it then will repeat the loop as long as a condition is true.

Syntax

```
do {  
    code to be executed;  
}
```

```
while (condition);
```

Example

The following example will increment the value of i at least once, and it will continue incrementing the variable i as long as it has a value of less than 10 –

```
<html><body><?php  
    $i = 0;  
    $num = 0;  
    do {  
        $i++;  
    }  
    while( $i < 10 );  
    echo ("Loop stopped at i = $i" );  
    ?>  
</body>  
</html>
```

This will produce the following result –

Loop stopped at i = 10

The foreach loop statement

The foreach statement is used to loop through arrays. For each pass the value of the current array element is assigned to \$value and the array pointer is moved by one and in the next pass next element will be processed.

Syntax

```
foreach (array as value) {  
    code to be executed;  
}
```

Example

Try out following example to list out the values of an array.

```
<html>  
<body>  
    <?php  
        $array = array( 1, 2, 3, 4, 5);  
        foreach( $array as $value ) {  
            echo "Value is $value <br />";  
        }  
    ?>  
</body>  
</html>
```

This will produce the following result –

Value is 1

Value is 2

Value is 3

Value is 4

Value is 5

The break statement

The PHP **break** keyword is used to terminate the execution of a loop prematurely. The **break** statement is situated inside the statement block. It gives you full control and whenever you want to exit from the loop you can come out. After coming out of a loop immediate statement to the loop will be executed.

Example

In the following example condition test becomes true when the counter value reaches 3 and loop terminates.

```
<html>  
<body>  
    <?php  
        $i = 0;  
        while( $i < 10) {  
            $i++;  
            if( $i == 3 ) break;  
        }  
        echo ("Loop stopped at i = $i");  
    ?>  
</body>  
</html>
```

This will produce the following result – Loop stopped at i = 3

The continue statement

The PHP **continue** keyword is used to halt the current iteration of a loop but it does not terminate the loop. Just like the **break** statement the **continue** statement is situated inside the statement block containing the code that the loop executes, preceded by a conditional test. For the pass encountering **continue** statement, rest of the loop code is skipped and next pass starts.

Example

In the following example loop prints the value of array but for which condition becomes true it just skip the code and next value is printed.

```
<html>
<body>
    <?php
        $array = array( 1, 2, 3, 4, 5);
        foreach( $array as $value ) {
            if( $value == 3 )continue;
            echo "Value is $value <br />";
        }
    ?>
</body>
</html>
```

This will produce the following result –

Value is 1

Value is 2

Value is 4

Value is 5

PHP Arrays

An array is a data structure that stores one or more similar type of values in a single value. For example, if you want to store 100 numbers then instead of defining 100 variables it's easy to define an array of 100 lengths. There are three different kind of arrays and each array value is accessed using an ID c which is called array index.

Numeric array: An array with a numeric index. Values are stored and accessed in linear fashion.

Associative array: An array with strings as index. This stores element values in association with key values rather than in a strict linear index order.

Multidimensional array: An array containing one or more arrays and values are accessed using multiple indices

NOTE – Built-in array functions is given in function reference [PHP Array Functions](#)

PHP Strings

They are sequences of characters, like "PHP supports string operations".

NOTE – Built-in string functions is given in function reference [PHP String Functions](#)

Following are valid examples of string

```
$string_1 = "This is a string in double quotes";
```

```
$string_2 = "This is a somewhat longer, singly quoted string";
```

```
$string_39 = "This string has thirty-nine characters";
```

```
$string_0 = ""; // a string with zero characters
```

Singly quoted strings are treated almost literally, whereas doubly quoted strings replace variables with their values as well as specially interpreting certain character sequences.

```
<?php
    $variable = "name";
    $literally = 'My $variable will not print!\n';
```

```

print($literally);
print "<br />";
$literally = "My $variable will print!\n";
print($literally);
?>

```

This will produce the following result –

My \$variable will not print!\n

My name will print!\n

String Concatenation Operator

To concatenate two string variables together, use the dot (.) operator –

```

<?php
$string1="Hello World";
$string2="1234";
echo $string1 . " " . $string2;
?>

```

This will produce the following result –

Hello World 1234

If we look at the code above, you see that we used the concatenation operator two times. This is because we had to insert a third string.

Between the two string variables we added a string with a single character, an empty space, to separate the two variables.

Using the strlen() function

The strlen() function is used to find the length of a string.

Let's find the length of our string "Hello world!" –

```

<?php
echo strlen("Hello world!");
?>

```

This will produce: 12

The length of a string is often used in loops or other functions, when it is important to know when the string ends. (i.e. in a loop, we would want to stop the loop after the last character in the string)

Using the strpos() function

The **strpos()** function is used to search for a string or character within a string.

If a match is found in the string, this function will return the position of the first match. If no match is found, it will return FALSE.

Let's see if we can find the string "world" in our string –

```

<?php
echo strpos("Hello world!","world");
?>

```

This will produce the following result – 6

As you see the position of the string "world" in our string is position 6. The reason that it is 6, and not 7, is that the first position in the string is 0, and not 1.

PHP - GET & POST Methods

There are two ways the browser client can send information to the web server.

The GET Method

The POST Method

Before the browser sends the information, it encodes it using a scheme called URL encoding. In this scheme, name/value pairs are joined with equal signs and different pairs are separated by the ampersand.

name1=value1&name2=value2&name3=value3

Spaces are removed and replaced with the + character and any other nonalphanumeric characters are replaced with a hexadecimal values. After the information is encoded it is sent to the server.

The GET Method: The GET method sends the encoded user information appended to the page request. The page and the encoded information are separated by the ? character.

http://www.test.com/index.htm?name1=value1&name2=value2

The GET method produces a long string that appears in your server logs, in the browser's Location: box.

The GET method is restricted to send upto 1024 characters only.

Never use GET method if you have password or other sensitive information to be sent to the server.

GET can't be used to send binary data, like images or word documents, to the server.

The data sent by GET method can be accessed using QUERY_STRING environment variable.

The PHP provides \$_GET associative array to access all the sent information using GET method.

Try out following example by putting the source code in test.php script.

```
<?php
if( $_GET["name"] || $_GET["age"] ) {
    echo "Welcome ". $_GET['name']. "<br />";
    echo "You are ". $_GET['age']. " years old.";
    exit();
}
?>
<html><body>
    <form action = "<?php $_PHP_SELF ?>" method = "GET">
    Name: <input type = "text" name = "name" />
    Age: <input type = "text" name = "age" />
    <input type = "submit" />
</form>
</body>
</html>
```

It will produce

A screenshot of a web browser displaying a simple form. The form has two text input fields. The first field is preceded by the label 'Name:' and the second by 'Age:'. To the right of the 'Age' field is a button labeled 'Submit'.

The POST Method: The POST method transfers information via HTTP headers. The information is encoded as described in case of GET method and put into a header called QUERY_STRING.

The POST method does not have any restriction on data size to be sent.

The POST method can be used to send ASCII as well as binary data.

The data sent by POST method goes through HTTP header so security depends on HTTP protocol. By using Secure HTTP you can make sure that your information is secure.

The PHP provides \$_POST associative array to access all the sent information using POST method.

Try out following example by putting the source code in test.php script.

```
<?php
if( $_POST["name"] || $_POST["age"] ) {
    if (preg_match("/^[^A-Za-z'-]"/,$_POST['name'])) {
        die ("invalid name and name should be alpha");
    }
    echo "Welcome ". $_POST['name']. "<br />";
    echo "You are ". $_POST['age']. " years old.";
    exit();
}
?>
```

```

<html><body>
    <form action = "<?php $_PHP_SELF ?>" method = "POST">
        Name: <input type = "text" name = "name" />
        Age: <input type = "text" name = "age" />
        <input type = "submit" />
    </form>
</body>
</html>

```

It will produce the following result –



The \$_REQUEST variable

The PHP \$_REQUEST variable contains the contents of both \$_GET, \$_POST, and \$_COOKIE. We will discuss \$_COOKIE variable when we will explain about cookies.

The PHP \$_REQUEST variable can be used to get the result from form data sent with both the GET and POST methods.

Try out following example by putting the source code in test.php script.

```

<?php if( $_REQUEST["name"] || $_REQUEST["age"] ) {
    echo "Welcome ". $_REQUEST['name']. "<br />";
    echo "You are ". $_REQUEST['age']. " years old.";
    exit();
}
?>

```

```

<html><body>
    <form action = "<?php $_PHP_SELF ?>" method = "POST">
        Name: <input type = "text" name = "name" />
        Age: <input type = "text" name = "age" />
        <input type = "submit" />
    </form>
</body>
</html>

```

Here \$_PHP_SELF variable contains the name of self script in which it is being called.

It will produce the following result –



PHP - File Inclusion

You can include the content of a PHP file into another PHP file before the server executes it. There are two PHP functions which can be used to included one PHP file into another PHP file.

The include() Function

The require() Function

This is a strong point of PHP which helps in creating functions, headers, footers, or elements that can be reused on multiple pages. This will help developers to make it easy to change the layout of complete website with minimal effort. If there is any change required then instead of changing thousands of files just change included file.

The include() Function

The include() function takes all the text in a specified file and copies it into the file that uses the include function. If there is any problem in loading a file then the **include()** function generates a warning but the script will continue execution.

Assume you want to create a common menu for your website. Then create a file menu.php with the following content.

```
<a href="http://www.tutorialspoint.com/index.htm">Home</a> -  
<a href="http://www.tutorialspoint.com/ebxml">ebXML</a> -  
<a href="http://www.tutorialspoint.com/ajax">AJAX</a> -  
<a href="http://www.tutorialspoint.com/perl">PERL</a> <br />
```

Now create as many pages as you like and include this file to create header. For example now your test.php file can have following content.

```
<html>  
  <body>  
    <?php include("menu.php"); ?>  
    <p>This is an example to show how to include PHP file!</p>  
  </body>  
</html>
```

It will produce the following result –



The require() Function

The require() function takes all the text in a specified file and copies it into the file that uses the include function. If there is any problem in loading a file then the **require()** function generates a fatal error and halt the execution of the script.

So there is no difference in require() and include() except they handle error conditions. It is recommended to use the require() function instead of include(), because scripts should not continue executing if files are missing or misnamed.

You can try using above example with require() function and it will generate same result. But if you will try following two examples where file does not exist then you will get different results.

```
<html><body>  
  <?php include("xxmenu.php"); ?>  
  <p>This is an example to show how to include wrong PHP file!</p>  
</body>  
</html>
```

This will produce the following result –

This is an example to show how to include wrong PHP file!

Now lets try same example with require() function.

```
<html>  
  <body>  
    <?php require("xxmenu.php"); ?>  
    <p>This is an example to show how to include wrong PHP file!</p>  
  </body>  
</html>
```

This time file execution halts and nothing is displayed.

NOTE – You may get plain warning messages or fatal error messages or nothing at all. This depends on your PHP Server configuration.

PHP Cookies

Cookies are text files stored on the client computer and they are kept of use tracking purpose. PHP transparently supports HTTP cookies.

There are three steps involved in identifying returning users –

Server script sends a set of cookies to the browser. For example, name, age, or identification number etc.

Browser stores this information on local machine for future use.

When next time browser sends any request to web server then it sends those cookies information to the server and server uses that information to identify the user.

Setting Cookies with PHP

PHP provided **setcookie()** function to set a cookie. This function requires upto six arguments and should be called before <html> tag. For each cookie this function has to be called separately.

setcookie(name, value, expire, path, domain, security);

Here is the detail of all the arguments –

Name: This sets the name of the cookie and is stored in an environment variable called HTTP_COOKIE_VARS. This variable is used while accessing cookies.

Value: This sets the value of the named variable and is the content that you actually want to store.

Expiry: This specify a future time in seconds since 00:00:00 GMT on 1st Jan 1970. After this time cookie will become inaccessible. If this parameter is not set then cookie will automatically expire when the Web Browser is closed.

Path: This specifies the directories for which the cookie is valid. A single forward slash character permits the cookie to be valid for all directories.

Domain: This can be used to specify the domain name in very large domains and must contain at least two periods to be valid. All cookies are only valid for the host and domain which created them.

Security: This can be set to 1 to specify that the cookie should only be sent by secure transmission using HTTPS otherwise set to 0 which mean cookie can be sent by regular HTTP.

Following example will create two cookies **name** and **age** these cookies will be expired after one hour.

```
<?php
    setcookie("name", "John Watkin", time()+3600, "/", "", 0);
    setcookie("age", "36", time()+3600, "/", "", 0);
?>
<html>
    <head>
        <title>Setting Cookies with PHP</title>
    </head>
    <body>
        <?php echo "Set Cookies"?>
    </body>
</html>
```

Accessing Cookies with PHP

PHP provides many ways to access cookies. Simplest way is to use either `$_COOKIE` or `$HTTP_COOKIE_VARS` variables. Following example will access all the cookies set in above example.

```
<html>
  <head>
    <title>Accessing Cookies with PHP</title>
  </head>
  <body>
    <?php
      echo $_COOKIE["name"]. "<br />";
      /* is equivalent to */
      echo $HTTP_COOKIE_VARS["name"]. "<br />";

      echo $_COOKIE["age"] . "<br />";

      /* is equivalent to */
      echo $HTTP_COOKIE_VARS["age"] . "<br />";
    ?>
  </body>
</html>
```

You can use **isset()** function to check if a cookie is set or not.

```
<html>
  <head>
    <title>Accessing Cookies with PHP</title>
  </head>
  <body>
    <?php
      if( isset($_COOKIE["name"]))
        echo "Welcome " . $_COOKIE["name"] . "<br />";
      else
        echo "Sorry... Not recognized" . "<br />";
    ?>
  </body>
</html>
```

Deleting Cookie with PHP

Officially, to delete a cookie you should call `setcookie()` with the name argument only but this does not always work well, however, and should not be relied on.

It is safest to set the cookie with a date that has already expired –

```
<?php
  setcookie( "name", "", time()- 60, "/", "", 0);
  setcookie( "age", "", time()- 60, "/", "", 0);
?>
<html>
  <head>
    <title>Deleting Cookies with PHP</title>
  </head>
  <body>
    <?php echo "Deleted Cookies" ?>
```

</body>

</html>

PHP - Sessions

An alternative way to make data accessible across the various pages of an entire website is to use a PHP Session.

A session creates a file in a temporary directory on the server where registered session variables and their values are stored. This data will be available to all pages on the site during that visit.

The location of the temporary file is determined by a setting in the **php.ini** file called **session.save_path**. Before using any session variable make sure you have setup this path.

When a session is started following things happen –

PHP first creates a unique identifier for that particular session which is a random string of 32 hexadecimal numbers such as 3c7foj34c3jj973hjkop2fc937e3443.

A cookie called **PHPSESSID** is automatically sent to the user's computer to store unique session identification string.

A file is automatically created on the server in the designated temporary directory and bears the name of the unique identifier prefixed by sess_ ie sess_3c7foj34c3jj973hjkop2fc937e3443.

When a PHP script wants to retrieve the value from a session variable, PHP automatically gets the unique session identifier string from the PHPSESSID cookie and then looks in its temporary directory for the file bearing that name and a validation can be done by comparing both values.

A session ends when the user loses the browser or after leaving the site, the server will terminate the session after a predetermined period of time, commonly 30 minutes duration.

Starting a PHP Session

A PHP session is easily started by making a call to the **session_start()** function. This function first checks if a session is already started and if none is started then it starts one. It is recommended to put the call to **session_start()** at the beginning of the page.

Session variables are stored in associative array called **\$_SESSION[]**. These variables can be accessed during lifetime of a session.

The following example starts a session then register a variable called **counter** that is incremented each time the page is visited during the session.

Make use of **isset()** function to check if session variable is already set or not. Put this code in a test.php file and load this file many times to see the result:

```
<?php
    session_start();
    if( isset( $_SESSION['counter'] ) ) {
        $_SESSION['counter'] += 1;
    }else {
        $_SESSION['counter'] = 1;
    }
    $msg = "You have visited this page ". $_SESSION['counter'];
    $msg .= "in this session.";
?>
<html>
<head>
    <title>Setting up a PHP session</title>
</head>
<body>
    <?php echo ( $msg ); ?>
</body>
```


</html>

It will produce the following result –

You have visited this page 1 in this session.

Destroying a PHP Session

A PHP session can be destroyed by **session_destroy()** function. This function does not need any argument and a single call can destroy all the session variables. If you want to destroy a single session variable then you can use **unset()** function to unset a session variable.

Here is the example to unset a single variable –

```
<?php
    unset($_SESSION['counter']);
?>
```

Here is the call which will destroy all the session variables –

```
<?php
    session_destroy();
?>
```

WEB PROGRAMMING MODULE